

499CPE: Project (2)

A PROJECT REPORT
PREPARED IN PARTIAL FULFILLMENT OF THE
REQUIREMENT FOR THE DEGREE OF
BACHELOR IN
COMPUTER ENGINEERING

IoT-Based Smart Water Grid Management System with AI Predictive Analytics

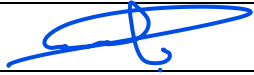
Abdulelah Hadi Alsalem	443801391
Abdulrahman Mohammed Alshehri	441808235
Basil Saad Alqahtani	443802933
Faisal Ali Asiri	443801580
Omar Mohammed Asiri	443801112

Supervisor

Dr. Amir Alrashid Abdalla Gasouma

BONA FIDE CERTIFICATE

This is to certify that this project report entitled “**IoT-Based Smart Water Grid Management System with AI Predictive Analytics**” is a bona fide work of **Abdulelah Hadi Alsalem, Abdulrahman Mohammed Alshehri, Basil Saad Alqhtani, Faisal Ali Asiri and Omar Mohammed Asiri** who carried out the project work under my supervision.

Student Name	Role	Signature
Abdulelah Hadi Alsalem	Team Member	
Abdulrahman Mohammed Alshehri	Team Member	
Basil Saad Alqahtani	Team Member	
Faisal Ali Asiri	Team Leader	
Omar Mohammed Asiri	Team Member	

Dr. Mohammad
Hassan Alhefdi

Head of Computer
Engineering Department

Dr. Amir Alrashid
Abdalla Gasouma

Faculty Supervisor

ACKNOWLEDGEMENT

We record heartfelt thanks and gratitude to Dr. Ayman Qahmash, Dean, College of Computer Science, King Khalid University (KKU) and Dr. Mohammad Alhefdi, Assistant Professor and Head, Department of Computer Engineering, College of Computer Science, KKU for providing us an opportunity to carry out this project.

We thank our project supervisor, Dr. Amir Alrashid, for providing valuable guidance and help in doing the project work.

We also thank all faculty members of Department of Computer Engineering, College of Computer Science, KKU for their support during our study and project work.

ABSTRACT

Water distribution networks in rapidly urbanizing regions face persistent challenges including undetected leakage, inefficient resource allocation, and limited real-time monitoring capabilities. These issues are particularly pressing in Saudi Arabia, where freshwater scarcity and the national Vision 2030 sustainability agenda demand modern, data-driven infrastructure solutions.

This project addresses the specific challenges of real-time water monitoring, automated distribution control, and predictive demand forecasting in a closed-loop water grid. The system targets three functional gaps: lack of continuous sensing across distribution zones, absence of centralized remote control, and the inability to anticipate consumption before waste occurs.

The proposed solution integrates three ESP32 microcontroller nodes equipped with flow, ultrasonic level, and TDS water quality sensors, connected to a Firebase Realtime Database cloud backend. A real-time HTML dashboard provides remote monitoring and actuator control, while an XGBoost machine learning model — trained on a 90-day synthetic digital twin dataset — forecasts hourly water demand. A mass-balance leakage detection algorithm compares company-side and customer-side flow measurements to identify network faults.

Feasibility was validated through hardware testing, cloud connectivity verification, and AI model evaluation. All sensors operated within specified tolerances following empirical calibration, dashboard latency averaged 1.1 seconds, and the XGBoost model achieved an R^2 score of 0.94 with a mean absolute error of 0.031 L/hour.

The system provides a low-cost, scalable platform for smart water grid management, demonstrating practical integration of IoT sensing, cloud analytics, and AI-driven forecasting in direct alignment with Saudi Arabia's National Water Strategy 2030.

Table of Contents

BONA FIDE CERTIFICATE	1
ACKNOWLEDGEMENT	2
ABSTRACT	3
Table of Contents	4
List of Tables.....	7
List of Figures	8
Chapter 1. INTRODUCTION	9
1.1. Overview	9
1.2. Project Motivations	9
1.3. Problem Statement	10
1.4. Project Objectives.....	10
1.5. Project Management	11
1.6. Background and Relevant Work	12
1.7. Significance	13
Chapter 2. SYSTEM DESIGN ALTERNATIVES.....	14
2.1. Design Requirements	14
2.2. Design Alternatives.....	14
2.2.1. Design Alternative 1.....	15
2.2.2. Design Alternative 2.....	16
2.3. Design Alternatives Evaluation Criteria	16
2.3.1. Decision Matrix	17
2.3.2. Selected Design Alternative	17
2.4. System Requirements.....	18
2.4.1. Functional Requirements.....	19
2.4.2. Hardware Requirements	19
2.4.3. Software Requirements	20
2.5. Realistic Constraints	20
2.5.1. Economic Constraints.....	20
2.5.2. Environmental Constraints	20

2.5.3.	Ethical Constraints	21
2.5.4.	Health and Safety Constraints	21
2.5.5.	Social Constraints.....	21
2.5.6.	Physical Constraints	21
2.6.	Standards to be Used.....	21
2.7.	Cost Analysis.....	22
Chapter 3.	SYSTEM DESIGN AND IMPLEMENTATION.....	23
3.1.	Detailed Design Description	23
3.1.1.	Overall System Architecture	24
3.1.2.	Software Architecture.....	25
3.1.3.	Hardware Architecture	30
3.1.4.	System Data Flow and Processing	31
3.2.	Modeling and Simulation.....	32
3.2.1.	Procedural Steps for Compilation	32
3.2.2.	Simulation/Prototype/Mathematical Model	37
3.2.3.	Deployment Steps.....	40
3.3.	Hardware Implementation	44
3.3.1.	Fabrication Steps	44
3.3.2.	Implementation Steps	47
3.3.3.	Implementation Demo	53
Chapter 4.	SYSTEM TESTING AND EVALUATION	55
4.1.	Verification Test Description.....	55
4.1.1.	Hardware Sensor and Actuator Testing.....	55
4.1.2.	Cloud Dashboard and Firebase Connectivity Testing	56
4.1.3.	AI Predictive Model Testing	59
4.2.	Results and Analysis.....	59
4.2.1.	Results	59
4.2.2.	Analysis	60
4.3.	Discussions	61
Chapter 5.	CONCLUSIONS AND RECOMENDATIONS	63
5.1.	Conclusions	63
5.2.	Recommendations	64
References.		66

Appendix.67

List of Tables

Table 1.1 GANTT Chart for semester 1 work.....	11
Table 1.2 GANTT Chart for semester 2 work.....	11
Table 2.1 Design Requirements Table.	14
Table 2.2 Decision Matrix's Table.	17
Table 2.3 Functional requirements.....	19
Table 2.4 Hardware Requirements.....	20
Table 2.5 Software Requirements.	20
Table 2.6 Used Standards.....	22
Table 2.7 Cost Analysis.....	22
Table 3.1 GPIO pin assignments across all three nodes.....	33
Table 3.2 Firebase database paths accessed by each node.	35
Table 3.3 Database fields.	43
Table 3.4 Dashboard checks.....	44
Table 4.1 Hardware Sensor and Actuator Verification Results.....	56
Table 4.2 Cloud Dashboard and Firebase Connectivity Test Results.	56
Table 4.3 XGBoost AI Model Accuracy Metrics.....	59
Table 4.4 Summary of System Test Results Against Functional Requirements.	60

List of Figures

Figure 1.1 Fishbone diagram for problem statement.....	10
Figure 2.1 Design Alternative 1	15
Figure 2.2 Design Alternative 2	16
Figure 2.3 Selected Design Alternative.....	18
Figure 3.1 General Block Diagram of the project	24
Figure 3.2 System-Level Software Architecture	25
Figure 3.3 Node 1 (Company Side) Software Flowchart	26
Figure 3.4 Node 2 / Node 3 (Customer Side) Software Flowchart	27
Figure 3.5 Company Dashboard Software Flowchart	28
Figure 3.6 Customer Dashboard Software Flowchart	29
Figure 3.7 Detailed Hardware Architecture of the System	30
Figure 3.8 Data Flow Diagram of the system.....	31
Figure 3.9 XGBoost Feature Importance Diagram	38
Figure 3.10 Time Series Benchmark Diagram	39
Figure 3.11 Model Accuracy Diagram	40
Figure 3.12 Database structure	42
Figure 3.13 G1/2 to 6mm fitting	45
Figure 3.14 6mm 1-to-2 splitter	46
Figure 3.15 G1/2 to G1/2 fitting.....	46
Figure 3.16 6mm manual leakage branch.....	46
Figure 3.17 Circuit Diagram of the System (Company Sides).....	47
Figure 3.18 Circuit Diagram of the System (Customer Sides).....	48
Figure 3.19 Water flow sensor	49
Figure 3.20 Solenoid valve.....	50
Figure 3.21 Ultrasonic sensor.....	51
Figure 3.22 TDS sensor.....	52
Figure 3.23 Water pump.....	53
Figure 3.24 Implementation Demo – top view.....	53
Figure 3.25 Implementation Demo – top view.....	54
Figure 3.26 Implementation Demo – customer 1 and leakage branch	54
Figure 4.1 Company dashboard running on normal conditions	57
Figure 4.2 dashboard running while leakage been detected.....	57
Figure 4.3 Firebase's database receives measures from the nodes	58
Figure 4.4 Customer dashboard running	58

Chapter 1.

INTRODUCTION

1.1. Overview

The IoT-Based Smart Water Grid Management System with AI Predictive Analytics is designed to improve water distribution efficiency through real-time monitoring and intelligent control. It utilizes ESP32 microcontrollers connected to flow, level, and quality sensors to collect accurate, continuous data. This data is transmitted to a cloud-based dashboard for visualization, storage, and remote management. The system integrates AI-based predictive modeling using classical machine learning algorithms such as XGBoost to forecast future water consumption. Predictions enable proactive decisions for optimizing water flow and reducing wastage. Automated valve control and alert mechanisms enhance reliability and response efficiency. The solution demonstrates seamless integration between IoT sensing, cloud connectivity, and AI analytics. It ensures equitable and sustainable water resource distribution across the grid. The project provides a scalable framework adaptable to urban, industrial, and agricultural environments. Ultimately, it supports smart city initiatives and contributes to sustainable resource management.

1.2. Project Motivations

Saudi Arabia faces increasing pressure on its limited freshwater resources due to rapid urbanization, population growth, and climatic challenges. The Kingdom's Vision 2030 emphasizes sustainability, digital transformation, and efficient resource utilization, making smart water management a national priority [1][2]. Traditional water networks often suffer from leakage, inefficient distribution, and lack of real-time monitoring [3]. Implementing an IoT-based smart water grid system addresses these challenges through continuous data collection, analytics, and intelligent control [4]. The integration of AI predictive models further enhances efficiency by forecasting demand and preventing wastage [5]. Such a system aligns with KSA's goals for smart infrastructure, sustainable cities, and technological innovation in water resource management [1]. This project contributes directly to achieving environmental sustainability and digital economy objectives under Vision 2030 [2].

1.3. Problem Statement

As shown in figure 1.1, Water distribution networks continue to experience excessive leakage and wastage due to limitations in monitoring, prediction, and data connectivity [3]. Traditional systems still depend on manual leak checks, which delay the detection of abnormal conditions and prevent timely intervention [3]. In addition, the absence of reliable forecasting tools leads to poor scheduling decisions and inefficient resource allocation across different zones of the grid. These challenges are further compounded by unstable network connectivity and the lack of a centralized cloud dashboard, resulting in fragmented visibility and limited real-time control. Together, these issues restrict the ability of operators to identify leaks early, anticipate demand, and maintain consistent communication between sensing nodes. Therefore, there is a growing need for an IoT-based smart water grid solution that integrates automated monitoring, AI-driven prediction, and robust cloud connectivity to minimize water losses and improve overall distribution efficiency in alignment with modern smart infrastructure goals [6].

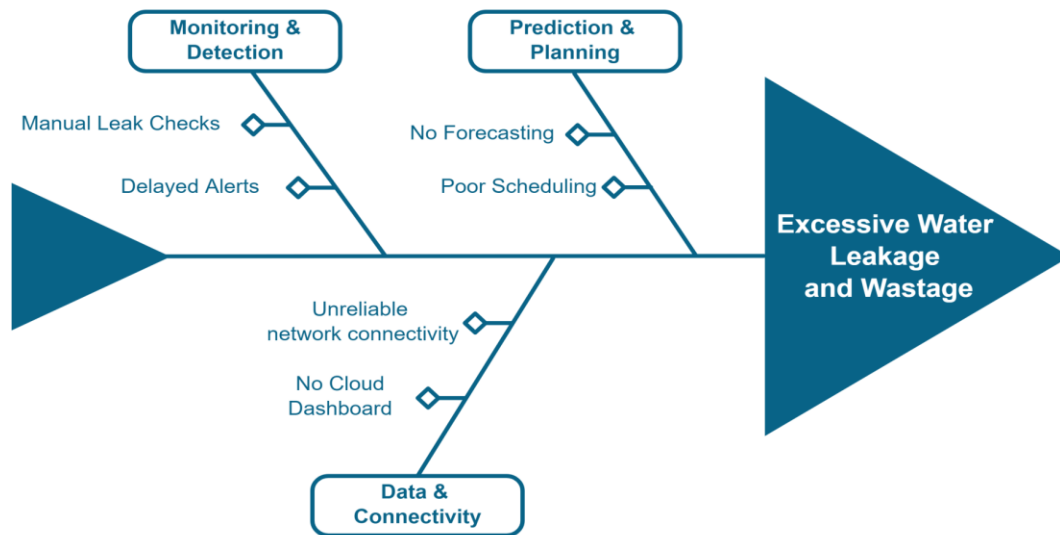


Figure 1.1 Fishbone diagram for problem statement

1.4. Project Objectives

The main aim of this project is to design and develop a Smart Water Management System that enhances water distribution through IoT-based monitoring, AI-driven prediction, and automated control. To achieve this aim, we plan to:

- A. Implement a smart water grid that continuously measures key parameters such as flow rate, and water level. And automate valve operations and generate timely alerts to improve operational efficiency and minimize water losses.
- B. Develop and integrate AI models capable of forecasting future water demand and identifying consumption patterns accurately. supporting proactive decision-making and improving distribution planning.
- C. Enable cloud-based visualization and system management, allowing remote monitoring, data logging, and control of the water grid through a centralized dashboard.

1.5. Project Management

Tasks per week	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Problem Identification																
Proposal Submission																
Literature Review																
Project Objectives																
Problem Statement																
Needs and Design Requirements																
Block Diagram																
Interim Presentation																
Chapter 1 Submission																
Chapter 2 Submission																

Table 1.1 GANTT Chart for semester 1 work.

Tasks per week	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Project 1 review																		
Models and block diagrams																		
Hardware Implementation																		
Dashboard & backend Implementation																		
AI data gathering & Training																		
Finish chapter 3																		
Interim Presentation																		
Testing and debugging																		
Finish chapter 4																		
Enclosure integration																		
Finish chapter 5																		
Final Presentation																		

Table 1.2 GANTT Chart for semester 2 work

1.6. Background and Relevant Work

Rapid urbanization, climatic stress, and limited freshwater resources have driven urgent demand for smarter water distribution and conservation solutions. Over the last five years research and pilot deployments have converged on integrated IoT + cloud/edge architectures (low-cost sensing nodes, gateways, MQTT/HTTP pipelines, time-series databases, dashboards) as practical approaches for real-time monitoring and control of water networks. Surveys and systematic reviews document mature IoT building blocks and recurring design patterns that inform sensor selection, network topology, and system scalability for prototype and field deployments [7].

Concurrently, machine learning has become central to short-term water-demand forecasting and anomaly detection. Classical ensemble methods such as XGBoost and Random Forest are repeatedly shown to be robust, fast to train, and highly competitive for hourly/daily demand prediction when combined with engineered time features and lagged consumption values; hybrid approaches (e.g., ML + LSTM or residual-correction using XGBoost) further improve accuracy in many recent studies. Ensemble and signal-analysis techniques are also used for leak detection and localization in real networks, often as AIoT pipelines combining edge sensing, local pre-processing, and centralized model inference [8].

Applied, low-cost hardware such as the ESP32 has been widely used in academic prototypes and practical projects for water-level, flow, and basic water-quality monitoring (TDS/turbidity/pH). Numerous open tutorials and projects demonstrate reliable ESP32 sensor interfacing and cloud connectivity (MQTT/HTTP) suitable for BSc-level prototyping and rapid iteration. These hardware patterns are frequently paired with dashboards (Grafana/Blynk) and lightweight databases for data logging and visualization [9].

Leak detection and anomaly-detection work (2020–2024) highlights the value of multi-sensor fusion (acoustic/hydrophone, flow differentials) and ensemble ML classifiers for high-fidelity detection in operational networks. Recent field studies show AIoT pipelines can reduce detection latency and improve localization accuracy, which is essential for operational savings and conservation [10].

Regionally, Saudi Arabia’s Vision 2030 and the National Water Strategy (2030) explicitly prioritize demand-management, modernization of water infrastructure, and innovation—creating a strong national mandate for scalable IoT and AI solutions that improve operational efficiency, reduce losses, and support sustainable water supply planning. This policy alignment motivates prototypes and pilot systems that directly target leakage reduction, demand forecasting, and data-driven distribution control [1][2].

Takeaways for this project: adopt a modular ESP32 → MQTT → time-series DB → dashboard architecture; collect high-quality timestamped data and engineer temporal features (hour/day/lag/rolling means); start with classical ML (XGBoost / Random Forest) for demand forecasting and combine with anomaly detectors for leak detection; deploy prediction as a lightweight REST/MQTT service (cloud or local edge) to enable automated valve control and alerts. These recommendations are supported by multiple 2020–2025 studies and practical prototypes [7].

1.7. Significance

This project holds strong significance in advancing sustainable water management through smart technology integration. By combining IoT sensing with AI-based predictive analytics, it enables efficient monitoring, control, and optimization of water distribution networks. The system supports real-time decision-making, reduces waste through early leak detection, and improves water resource allocation. Implementing this solution aligns directly with Saudi Arabia’s Vision 2030 goals for smart infrastructure, digital transformation, and environmental sustainability. Furthermore, the project provides a scalable and low-cost model suitable for municipalities, industries, and communities across the Kingdom.

Chapter 2.

SYSTEM DESIGN ALTERNATIVES

2.1. Design Requirements

The project design requires a reliable sensing and communication architecture that can accurately track water levels, flow rates, and usage trends in real time. It must include secure cloud connectivity, AI-based consumption prediction, and automated control logic to ensure efficient water distribution. For deployment in a range of Saudi Arabian locations, the design should also ensure scalability, low power consumption, and strong environmental durability. Table 2.1 provides an overview of both its design requirements and general design needs.

Design Need	Design Requirement
Able to monitor the System in Real-time	• Flow sensor • Level sensor • Quality sensor
Able to transmit data reliably	• Data communication protocols
Able to visualize and control the System remotely	• Cloud-based dashboard
Able to predict water consumption accurately	• AI model trained in historical sensor data
Able to process automatically	• Processing unit
Able to actuate efficiently	• Control valves

Table 2.1 Design Requirements Table.

2.2. Design Alternatives

Three design alternatives are considered: a basic sensor-only monitoring system with manual data review, a semi-automated IoT design with cloud dashboards and rule-based control, and the proposed advanced architecture integrating IoT sensing, automated actuation, and AI-based consumption prediction. These alternatives are evaluated in terms of cost, scalability, accuracy, and suitability for large-scale smart water management. The final proposed design offers the highest performance and future expansion capability while meeting national smart-infrastructure requirements.

2.2.1. Design Alternative 1

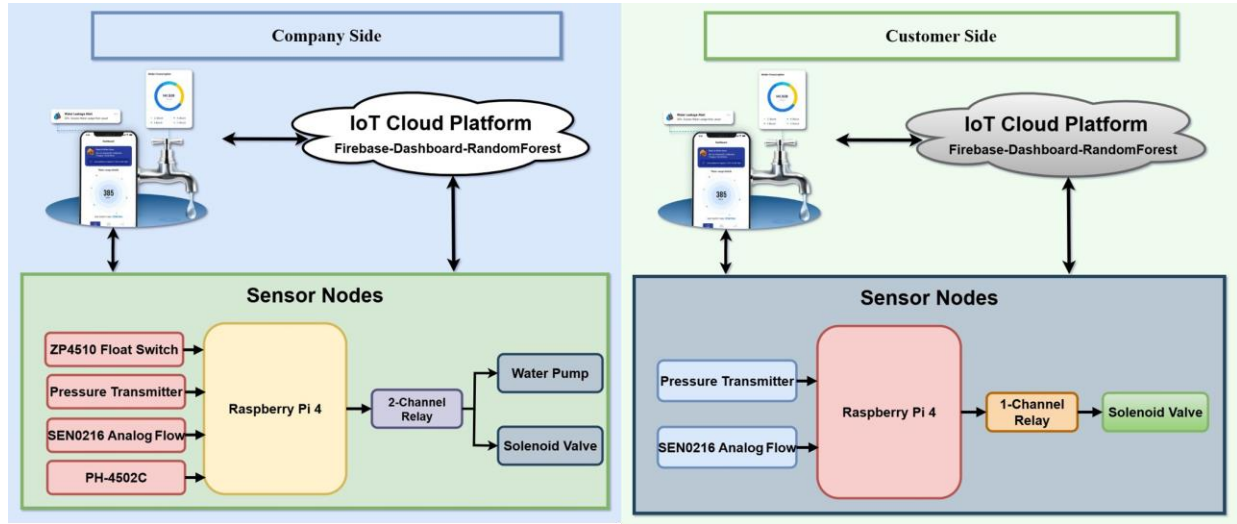


Figure 2.1 Design Alternative 1

Design Alternative 1 shown in figure 2.1 is a basic monitoring system that utilizes heterogeneous sensing components with a general-purpose processing platform. In this design, a Raspberry Pi collects data from various sensors, including a float switch, an analog flow sensor, and a pH sensor, along with additional sensing on the customer side. The data is transmitted to a cloud platform for storage and visualization through a simple dashboard, with limited analytical capability using a basic machine learning model. Control is implemented via relay modules to operate the water pump and the solenoid valve. While this design provides essential monitoring and control functionality, its use of less integrated components and limited intelligence results in lower efficiency and reduced scalability compared to more advanced solutions.

2.2.2. Design Alternative 2

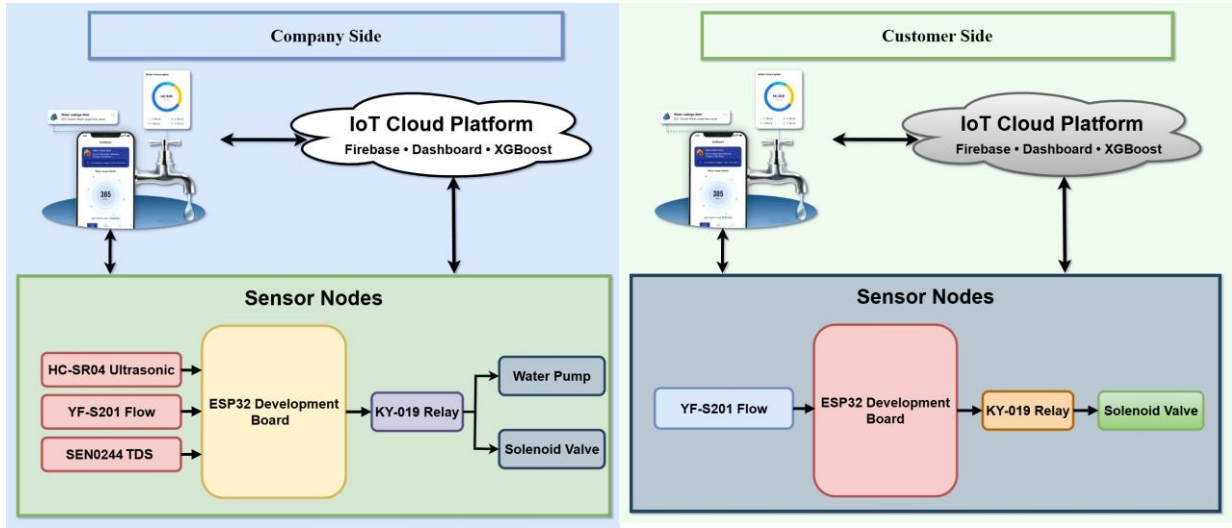


Figure 2.2 Design Alternative 2

Design Alternative 2 shown in figure 2.2 is the full proposed system integrating IoT sensing, cloud connectivity, automated actuation, and AI-driven water consumption prediction. This design enables intelligent decision-making, proactive distribution control, and early leak detection through real-time analytics and machine learning. It offers the highest efficiency, scalability, and alignment with Saudi Arabia’s smart-infrastructure and sustainability goals.

2.3. Design Alternatives Evaluation Criteria

To determine the most appropriate design alternative for the IoT-Based Smart Water Grid Management System, several evaluation criteria were defined to differentiate the proposed architectures. Initially, the raw advantages and disadvantages of each alternative were identified, comparing the simplicity and low cost of the basic sensor-only design, the improved visibility offered by the IoT-enabled monitoring design, and the comprehensive intelligence of the fully integrated IoT- and AI-driven system. To conduct a more structured assessment, key categories such as the previously mentioned design requirements. Along with cost, physical size, and environmental impact were established. Each design was evaluated against these criteria to determine its suitability for scalable and efficient water-grid management.

2.3.1. Decision Matrix

The design alternative evaluation criteria for our project are the design needs previously mentioned in 2.1 along with some criteria that will be explored further ahead. In table 2.2, we shall compare the design alternatives using a weighted approach to emphasize the importance of each requirement. We will use a scale of zero to two for evaluation where: 0: does not fit, 1: somehow fit and 2: totally fit to the design needs.

Requirements / Criteria		Design Alternative #1		Design Alternative #2	
Name	w	v	$w \times v$	v	$w \times v$
Able to monitor the System in Real-Time	0.1	2	0.2	2	0.2
Able to transmit data reliably	0.15	2	0.3	2	0.3
Able to visualize and control the System remotely	0.15	2	0.3	2	0.3
Able to predict water consumption accurately	0.2	1	0.2	2	0.4
Able to make decisions automatically	0.15	1	0.15	2	0.3
Able to actuate efficiently	0.1	2	0.2	2	0.2
Cost	0.05	0	0	2	0.10
Size	0.05	1	0.05	1	0.05
Environmental Impact	0.05	2	0.10	2	0.1
Total Points		13	1.5	17	1.95

Table 2.2 Decision Matrix's Table.

2.3.2. Selected Design Alternative

Evaluation Criteria showed that Alternative 2 is the best among the alternatives, because: -

- It was the only design with **full IoT, automation, and AI prediction capabilities**.
- **It had the best performance** in real-time monitoring, reliability, and remote control.

- **It is the most scalable and efficient** for smart-water management goals.
- It Supports **sustainability** through **optimized distribution and leak detection**.

Thus, as shown in figure 2.3, we used design alternative 2 that has an ESP32 controller, IoT sensing capability, cloud connectivity, automated actuation, and AI-driven water consumption prediction for our water grid implementation.

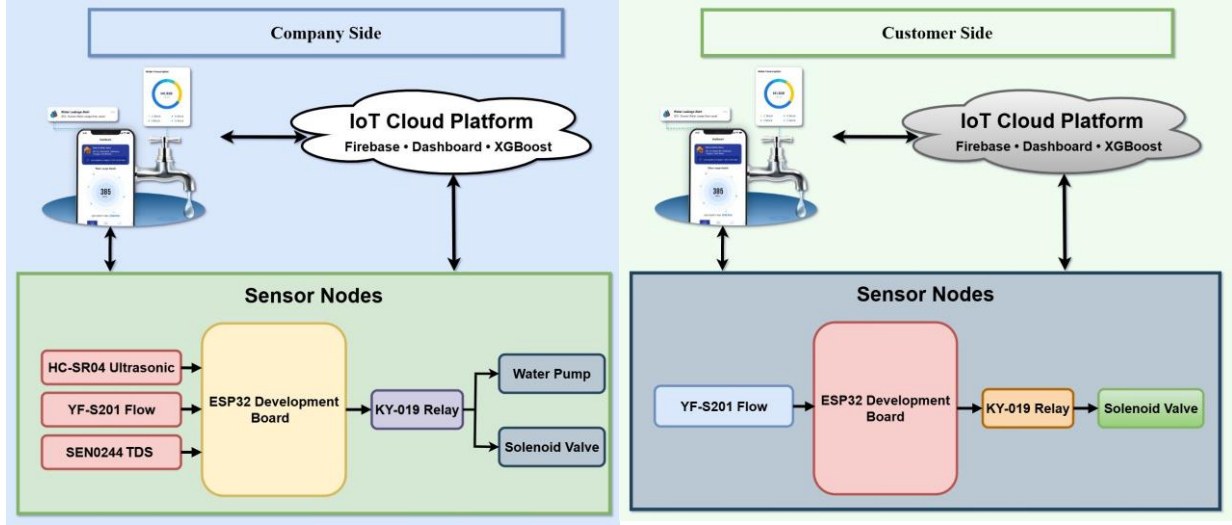


Figure 2.3 Selected Design Alternative

2.4. System Requirements

System requirements define the complete set of functional and non-functional requirements that the proposed IoT-Based Smart Water Grid Management System must satisfy stakeholder needs. These requirements serve as the foundation for system architecture, design, implementation, verification, and validation. They also act as a common reference point for communication among all technical and non-technical stakeholders involved in the project.

2.4.1. Functional Requirements

Functional Requirement	Item	Function
Real-time sensing of water parameters	Sensors (flow, level, quality)	Measure water variables continuously and send readings to ESP32
Data acquisition and processing	ESP32 Microcontroller	Collect sensor data, preprocess readings, and transmit to the cloud
Cloud connectivity and monitoring	Wi-Fi + Cloud Dashboard	Upload data, display real-time values, and provide remote user access
Predictive water consumption analysis	AI/ML Module (Cloud or Edge)	Predict future water demand using historical data and classical ML models
Automated valve control	Solenoid Valves + Relay Modules	Regulate water distribution based on predictions and system logic
Anomaly detection and alerts	ESP32 + Cloud Notification System	Identify leaks, abnormal flow, or sensor faults and notify users
Data logging and storage	Cloud Database	Store historical data for analytics, reporting, and model training
System scalability	Modular ESP32 nodes	Allow easy expansion to additional tanks, zones, or pipelines
Secure communication	TLS/HTTPS/MQTT Security	Protect transmitted data against unauthorized access
Power management	Power Supply Unit	Provide stable and efficient power for sensors, ESP32, and actuators

Table 2.3 Functional requirements.

2.4.2. Hardware Requirements

Component	Specification
ESP32 Development Board	240 MHz dual-core processor, built-in Wi-Fi & Bluetooth, multiple GPIO pins, ADC, UART, I ² C, SPI interfaces
Ultrasonic / Level Sensor	Non-contact distance measurement, suitable for water level sensing, operating voltage 3.3–5 V
Flow Sensor (YF-S201)	Hall-effect based water flow measurement, pulse output, operating voltage 5–18 V, suitable for per-zone flow monitoring
TDS / Turbidity Sensor	Water quality monitoring sensor, measures total dissolved solids or turbidity level, analog output compatible with MCU ADC
Solenoid Valve	Electrically controlled water valve, normally closed, suitable for automatic water flow control
Relay / MOSFET Driver	High-current switching module, isolates MCU from load, used to safely drive solenoid valves
Power Supply Unit	Regulated DC power supply providing stable voltage and current for MCU, sensors, and actuators
Enclosure & Mounting	Protective housing for electronics, provides mechanical stability and environmental protection
Wiring & Connectors	Insulated wires, terminal blocks, and connectors ensuring reliable electrical connections

Component	Specification
Breadboard	Used for circuit prototyping and assembly before final PCB implementation
Water Container	Stable water storage tank that is suitable for level and quality.
Water Pipes & Fittings	Plumbing components to distribute water flow and integrate sensors into the system
12V Water Pump (Non-Submersible)	External inline DC water pump, 12 V operation, self-priming or centrifugal type, used for water transfer.

Table 2.4 Hardware Requirements.

2.4.3. Software Requirements

Component	Specification
Arduino IDE	Used for programming and uploading firmware to the ESP32 microcontroller, supports necessary libraries for sensors, Wi-Fi, and cloud communication.
Python Environment (Collab / Python 3.10+)	Used for developing and training the AI-based water consumption prediction model, required libraries: pandas, NumPy, scikit-learn, matplotlib.
Firebase Cloud service	Enables real-time data gathering and streaming to dashboard.
AI Prediction Model	Trained on a digital twin of the gathered data, predicts the consumption based on the diurnal cycle.
HTML-Based Dashboard	Displays real-time sensor data, predictions, alerts, and system status to the user, supports API integration and data logging.

Table 2.5 Software Requirements.

2.5. Realistic Constraints

2.5.1. Economic Constraints

- The system must use low-cost sensors and ESP32 controllers due to limited budget.
- Cloud services and data storage should rely on free or low-cost platforms.
- The design must remain affordable for municipalities, farms, and households.

2.5.2. Environmental Constraints

- Not suitable for extreme temperatures above 55°C without extra cooling.
- Sensors exposed to water may suffer from corrosion or mineral buildup.

2.5.3. Ethical Constraints

- Water usage data must be protected and not misused.
- AI predictions must not mislead users or cause unfair water distribution.
- System design must follow IEEE ethical guidelines regarding safety and transparency.

2.5.4. Health and Safety Constraints

- Water and electricity must be isolated to prevent electrical hazards.
- Incorrect valve control can cause high pipe pressure or flooding.
- Only trained users should access the valve control dashboard.

2.5.5. Social Constraints

- The system must not interrupt users' daily water supply.

2.5.6. Physical Constraints

- ESP32 nodes, sensors, and valves must fit in compact installation spaces.
- Long wiring distances may cause signal loss and inaccurate readings.
- Power supply must be stable and protected from outdoor conditions.

2.6. Standards to be Used

The IoT-Based Smart Water Grid Management System uses multiple engineering and technological standards to guarantee reliable communication, secure data transfer, precise sensor interfacing plus efficient software development. Those standards in table 2.6 specify how the ESP32 nodes exchange data through Wi-Fi, how the sensors gather and send readings, how the cloud platform links to the AI prediction module but also how the whole system keeps its quality.

Category	Standard / Technology
Wi-Fi Network	• IEEE 802.11 b/g/n
Cloud Communication	• HTTPS / TLS 1.2 (RFC 5246)
Data Exchange Format	• JSON (RFC 8259 / ECMA-404)
Web Dashboard	• HTML5 / CSS3 / ECMAScript (W3C standards)
Embedded Firmware	• C/C++ (ISO/IEC 14882) via Arduino Framework
Electrical Safety	• IEC 62368-1

Table 2.6 Used Standards.

2.7. Cost Analysis

The table 2.7 below presents the cost of all project components, including the quantity of each item and the total estimated cost [1].

Component	Quantity	Cost
ESP32 Development Board	3	22 \$
Ultrasonic Level Sensor (HC-SR04)	3	3 \$
Water Flow Sensor (YF-S201)	3	6 \$
TDS / Turbidity Sensor	1	6 \$
Solenoid Valve	3	30 \$
Relay Module	6	4 \$
Power Supply Module	2	50 \$
Wiring & Connectors	assorted	14 \$
Voltage Transformer	1	20 \$
Breadboards / PCB	3	16 \$
Water Pump	1	30 \$
Water Container	1	2 \$
Water Pipes and Pipes Fitting	assorted	25 \$
Enclosure & Mounting Hardware	3	50 \$
Total Cost: 278 \$		

Table 2.7 Cost Analysis.

Chapter 3.

SYSTEM DESIGN AND IMPLEMENTATION

The proposed IoT-Based Smart Water Grid Management System with AI Predictive Analytics was implemented as an integrated hardware-software platform designed to monitor, analyze, and control water distribution in real time. The system combines sensing devices, communication modules, cloud services, and machine learning models into one intelligent architecture. The implementation focused on scalability, affordability, and ease of deployment using commercially available components such as ESP32 microcontrollers, sensors, relay modules, and cloud dashboards.

3.1. Detailed Design Description

This section provides a detailed description of the proposed IoT-Based Smart Water Grid Management System. The design is presented in a structured manner, starting from the overall system architecture and then moving to more detailed components.

The system is composed of three main parts: hardware, software, and cloud with AI-based analytics. The hardware layer includes sensors, microcontrollers, and actuators responsible for data acquisition and physical control. The software layer handles data processing, communication, and controlling logic. The cloud and AI layer is responsible for data storage, visualization, and predictive analysis.

To improve clarity, the design is divided into subsections, where each part of the system is explained in detail using block diagrams and data flow representations.

3.1.1. Overall System Architecture

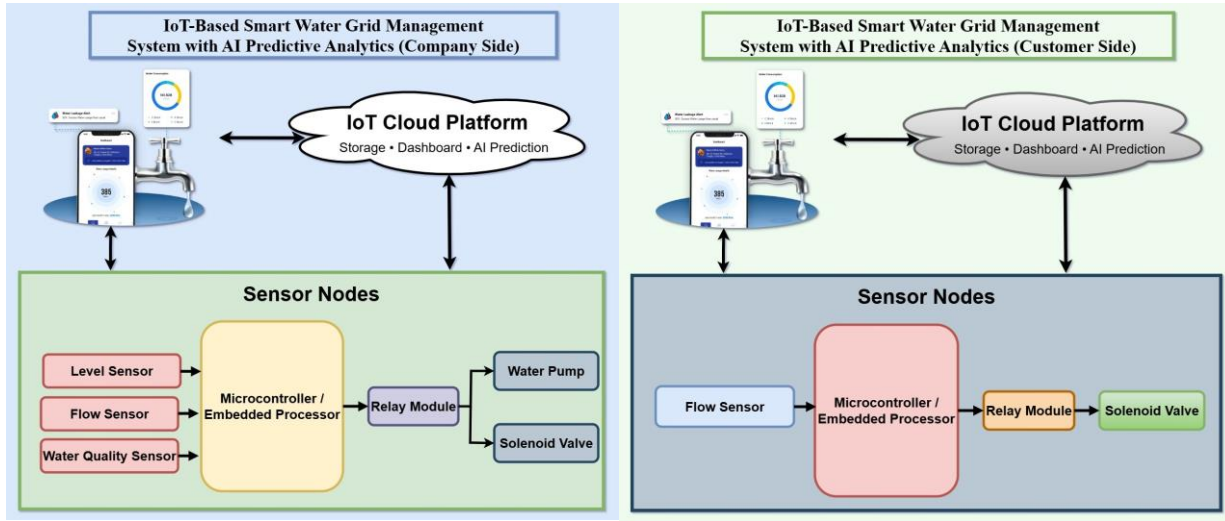


Figure 3.1 General Block Diagram of the project

The general block diagram shown in figure 3.1 illustrates the overall architecture of the IoT-Based Smart Water Grid Management System, showing the interaction between the company side, the customer side, and the IoT cloud platform.

The system consists of sensor nodes, ESP32 microcontrollers, a cloud platform, and actuator units. On the company side, sensors including level, flow, and water quality continuously collect real-time data. This data is processed by the microcontroller and transmitted to the IoT cloud platform, which provides data storage, dashboard visualization, and AI-based prediction.

Based on the analysis results, control signals are generated and sent back to the system to operate relay modules that control the water pump and solenoid valve, enabling automated water distribution.

On the customer side, a flow sensor is used to monitor water consumption, while a microcontroller manages data transmission and controls the solenoid valve for local water regulation.

Overall, the system operates as a closed-loop architecture that integrates real-time sensing, cloud-based analysis, and automated control to achieve efficient water management.

3.1.2. Software Architecture

This section describes the software architecture of the system through a series of flowcharts that illustrate the program logic governing each component. The software is organized into five distinct functional modules: the system-level communication architecture, the company-side ESP32 firmware (Node 1), the customer-side ESP32 firmware (Nodes 2 and 3), the company monitoring dashboard, and the customer billing portal. Each module is presented as a detailed flowchart followed by a description of its operational logic.

1. System-Level Software Overview

The system-level software architecture shown in figure 3.2 defines how all five software components communicate through the Firebase Realtime Database, which serves as the central communication hub. No direct device-to-device communication exists; all data exchange between nodes, dashboards, and the AI server is mediated through Firebase.

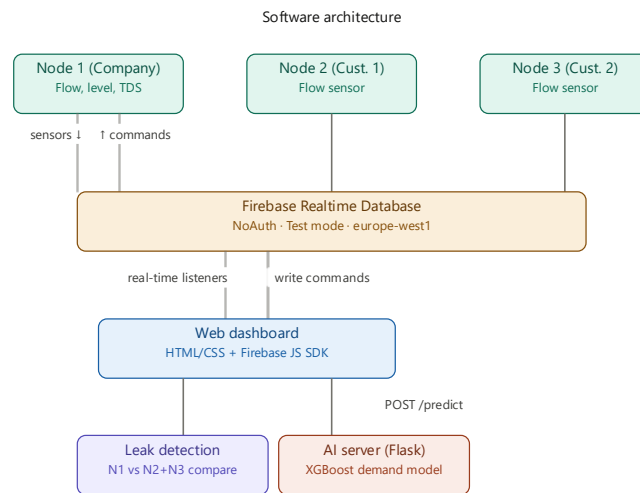


Figure 3.2 System-Level Software Architecture

As illustrated in the figure, the three ESP32 nodes continuously push sensor readings to Firebase and pull actuator command states on every loop iteration. The company dashboard establishes real-time listeners on all three node paths and writes actuator commands (valve toggle, pump toggle, consumption reset) back to Firebase. The customer dashboard listens only to the relevant customer node and performs billing calculations locally. The AI prediction server receives consumption data from the dashboard via HTTP POST requests and returns demand forecasts generated by the XGBoost model.

2. Node 1 Firmware — Company Side

As shown in figure 3.3, node 1 is the most complex firmware module, responsible for reading all company-side sensors (flow, ultrasonic level, and TDS water quality), controlling both the solenoid valve and the water pump, and managing consumption accumulation with hourly reset logic.

The `setup()` function initializes serial communication, configures GPIO pins for all sensors and actuators, establishes a Wi-Fi connection, initializes the Firebase client with NoAuth authentication, and attaches a hardware interrupt to the flow sensor pin (GPIO 27) to capture pulses via an ISR declared with the `IRAM_ATTR` attribute.

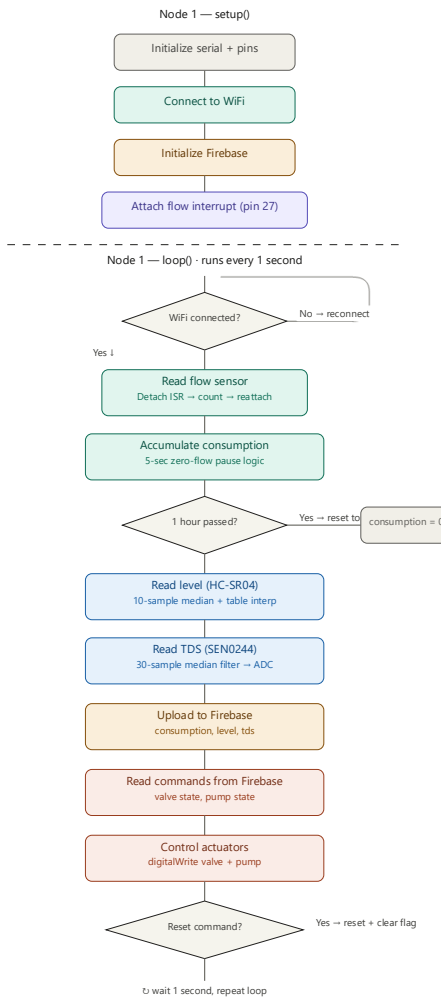


Figure 3.3 Node 1 (Company Side) Software Flowchart

The `loop()` function executes once per second and follows a sequential pipeline. First, the Wi-Fi connection status is verified, and reconnection is attempted if the link has dropped. Next, the flow sensor interrupt is temporarily detached, the accumulated pulse count is read and converted to liters using the

empirically calibrated factor of 225 pulses per liter, and the interrupt is re-attached. The consumption accumulator applies a 5-second zero-flow pause logic: if no flow is detected for five consecutive seconds, accumulation pauses while the hourly timer continues to run.

Following flow measurement, the ultrasonic level sensor acquires 10 distance samples at 50-millisecond intervals. The median of the sorted valid readings is interpolated against a pre-measured distance-to-volume calibration table and expressed as a percentage of tank capacity. The TDS sensor then acquires 20 ADC samples, extracts the median value, and converts it to parts per million using the manufacturer-specified third-order polynomial.

After all sensor readings are obtained, the node uploads the current consumption, level, and TDS values to Firebase. It then reads the actuator command states (valve and pump booleans) from Firebase and applies them to the corresponding relay output pins via `digitalWrite`. Finally, the node checks for a dashboard-issued consumption reset command; if the reset flag is true, the consumption counter is zeroed, the hourly timer is restarted, and the flag is immediately cleared to prevent repeated resets on subsequent loop iterations.

3. Nodes 2 and 3 Firmware — Customer Side

As shown in figure 3.4, nodes 2 and 3 follow the same architectural pattern as Node 1 but with reduced sensor and actuator scope. Each customer node interfaces with a single YF-S201 flow sensor and controls one normally closed solenoid valve through a relay module. The ultrasonic level sensor, TDS sensor, and water pump are absent from these nodes.

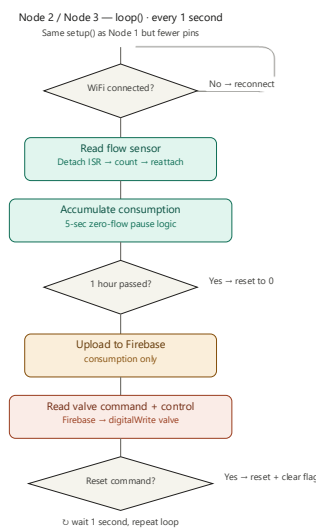


Figure 3.4 Node 2 / Node 3 (Customer Side) Software Flowchart

The loop structure mirrors Node 1: Wi-Fi connection verification, interrupt-based flow measurement with the same 225-pulse calibration factor, consumption accumulation with 5-second zero-flow detection, and hourly automatic reset. The key differences are that only consumption is uploaded to Firebase (no level or TDS fields), only the valve state is read from Firebase (no pump command), and the Firebase paths use their respective node prefixes (`/nodes/node2/` or `/nodes/node3/`). This modular design ensures that adding additional customer nodes requires only duplicating the firmware with an updated node path prefix.

4. Company Dashboard Software

The company dashboard shown in figure 3.5 is implemented as a self-contained HTML/JavaScript application utilizing the Firebase JS SDK (compatibility version 9) via CDN. Unlike the ESP32 firmware, which operates on a polling loop, the dashboard employs an event-driven architecture through Firebase `.on("value")` listeners that fire automatically upon each database state change.

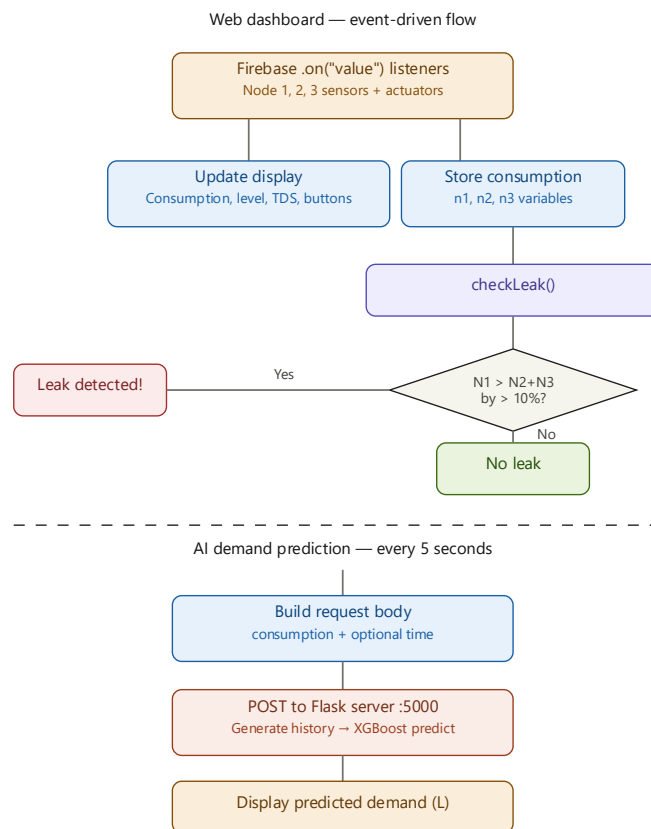


Figure 3.5 Company Dashboard Software Flowchart

Upon page load, the dashboard registers real-time listeners on all three node sensor paths (`/nodes/node1/sensors`, `/nodes/node2/sensors`, `/nodes/node3/sensors`). When any sensor value changes in

Firebase, the corresponding callback updates the HTML display elements and stores the latest consumption values in local JavaScript variables for use by the leakage detection algorithm.

The leakage detection module implements a mass-balance comparison between the company-side flow measurement (Node 1) and the aggregate customer-side measurements (Nodes 2 + 3). The algorithm computes the absolute percentage difference between these values. A difference below 5% is classified as normal operation, accommodating the inherent measurement uncertainty of the YF-S201 sensors. A difference between 5% and 15% triggers a warning status, while a difference exceeding 15% triggers an alert indicating significant leakage that requires immediate operational intervention.

Separately, the AI demand prediction module runs on a 5-second interval timer. On each cycle, it constructs a JSON request body containing the current Node 1 consumption value and optionally manual hour and day-of-week overrides. This payload is sent via HTTP POST to the Flask prediction server running on localhost:5000. The server generates synthetic historical features from the current consumption value and feeds them through the trained XGBoost model to produce an hourly demand forecast, which is returned to the dashboard and displayed in the prediction card.

5. Customer Dashboard Software

The customer dashboard shown in figure 3.6 provides a simplified, consumer-facing portal for viewing water consumption and calculating tiered billing. Its software flow is driven by user interaction rather than continuous monitoring.

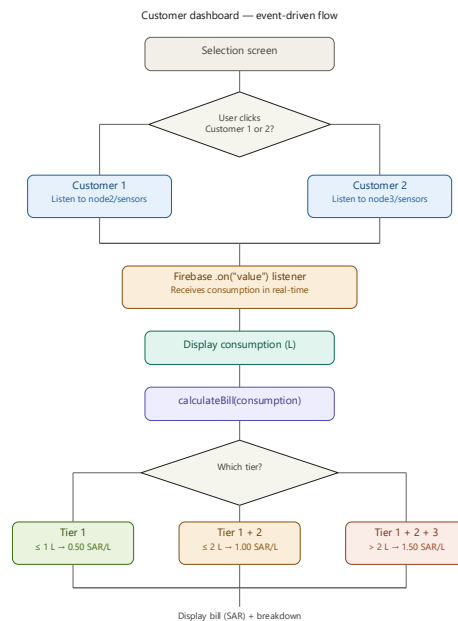


Figure 3.6 Customer Dashboard Software Flowchart

The application begins at a selection screen where the user chooses their account (Customer 1 or Customer 2). This selection determines the Firebase listener path: Customer 1 maps to /nodes/node2/sensors and Customer 2 maps to /nodes/node3/sensors. Upon selection, any previously active Firebase listener is detached to prevent stale data bindings, and a new .on("value") listener is attached to the selected node path.

Each time the listener fires with updated consumption data, the calculateBill() function computes the customer bill using a three-tier pricing structure. The first liter of consumption is billed at 0.50 SAR per liter, the second liter at 1.00 SAR per liter, and all consumption above 2 liters at 1.50 SAR per liter. The function returns both the total bill amount and a human-readable breakdown string showing the contribution of each tier. Both values are displayed in real time on the dashboard. A back button detaches the current Firebase listener, clears the customer state, and returns to the selection screen.

3.1.3. Hardware Architecture

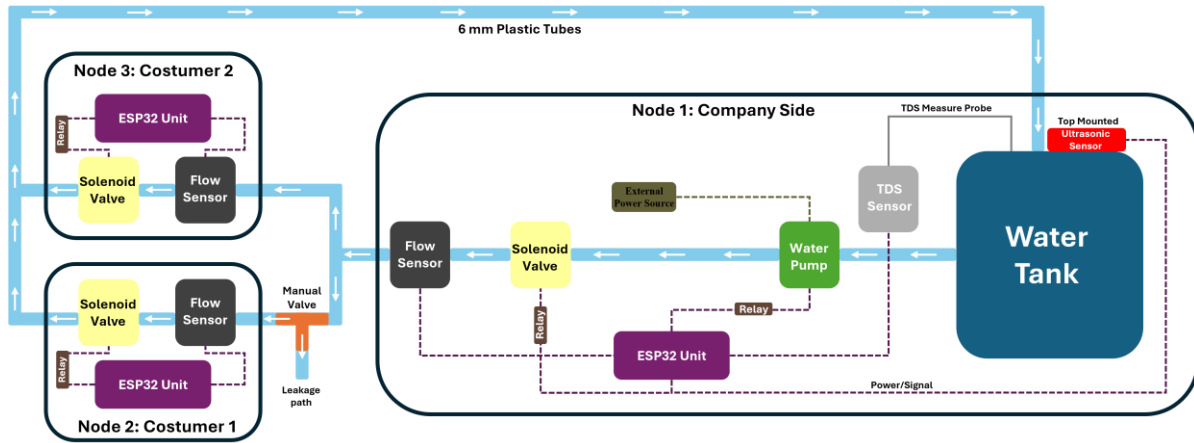


Figure 3.7 Detailed Hardware Architecture of the System

The detailed hardware architecture shown in figure 3.7 illustrates the physical implementation of the smart water grid system, including the company-side station and multiple customer nodes interconnected through a closed-loop water distribution network.

On the company side, the system is centered around a water tank equipped with an ultrasonic sensor for water level measurement and a TDS sensor for monitoring water quality. A water pump is used to drive water through the pipeline, while a main solenoid valve regulates the flow. Additionally, a flow sensor is installed in the main line to measure the overall water distribution.

All sensors and actuators on the company side are interfaced with an ESP32 microcontroller, which serves as the central control unit. The ESP32 processes sensor data and controls the water pump and solenoid valve through relay modules. An external power source is used to supply sufficient power for high-load components such as the pump.

On the customer side, the system consists of multiple distributed nodes, where each node includes a flow sensor, a solenoid valve, and an ESP32 unit. These nodes are connected to the main pipeline and are responsible for monitoring individual water consumption and enabling independent control of water flow for each customer.

The entire system is implemented as a closed-loop piping structure, allowing continuous water circulation back to the tank. This configuration ensures stable operation, accurate sensing, and effective control across both the company and customer sides of the network.

3.1.4. System Data Flow and Processing

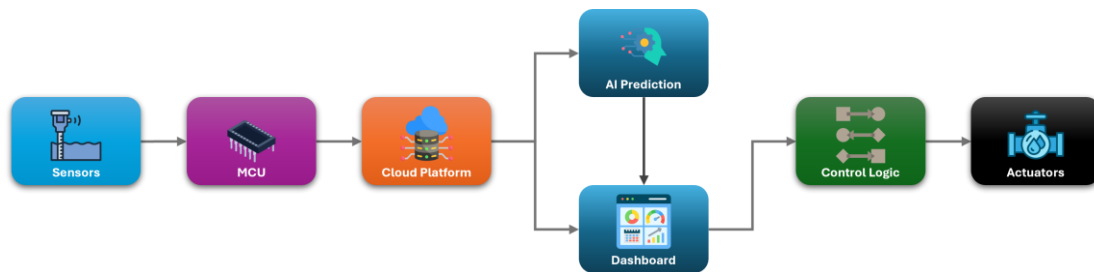


Figure 3.8 Data Flow Diagram of the system

The system data flow diagram shown in figure 3.8 illustrates how data moves through the IoT-Based Smart Water Grid Management System, from data acquisition to decision-making and control execution.

The process begins at the sensor layer, where physical parameters such as water level, flow rate, and water quality are continuously measured. These readings are transmitted to the ESP32 microcontroller, which performs initial data processing, including filtering and formatting.

The processed data is then sent to the cloud platform, where it is stored and made available for further analysis. Within the cloud layer, two main operations are performed: real-time visualization through the dashboard and predictive analysis using AI models.

The AI prediction module analyzes historical and real-time data to estimate future water consumption and detect abnormal conditions. At the same time, the dashboard provides users with live system status, alerts, and monitoring capabilities.

The outputs from both the AI module and the dashboard are passed to the control logic layer, which determines the appropriate system response based on predefined rules and system conditions.

Finally, control commands are sent to the actuator layer, including the water pump and solenoid valves, to regulate water flow and ensure efficient system operation.

3.2. Modeling and Simulation

3.2.1. Procedural Steps for Compilation

This section presents the essential code segments of the system, comprising three ESP32 firmware nodes and a web-based monitoring dashboard, organized by functional role. Logic common to all nodes is presented once; node-specific behavior is identified explicitly.

1. Required Libraries

Prior to compilation, three external libraries must be installed via the Arduino IDE Library Manager (Sketch → Include Library → Manage Libraries). These dependencies are common to all three nodes.

Listing 3.1 — Library includes (common to all nodes)

```
#include <WiFi.h>           // IEEE 802.11 Wi-Fi connectivity
#include <WiFiClientSecure.h> // TLS/SSL encrypted socket layer
#define ENABLE_DATABASE    // Restrict compilation to the database module only
#include <FirebaseClient.h> // Firebase Realtime Database client — Mobizt v2.2.9
```

The `ENABLE_DATABASE` preprocessor directive restricts the `FirebaseClient` library to the Realtime Database module, reducing the compiled binary size. `WiFiClientSecure` establishes a TLS-encrypted socket through which all Firebase communications are secured in transit.

GPIO pin assignments vary across nodes in accordance with each node's hardware role within the closed-loop distribution circuit, as summarized in Table 3.1.

Signal	Node 1 — Company	Node 2 — Customer 1	Node 3 — Customer 2
Flow sensor (YF-S201)	GPIO 27	GPIO 27	GPIO 27
Ultrasonic TRIG / ECHO	GPIO 5 / GPIO 18	—	—
TDS sensor (SEN0244)	GPIO 35	—	—
Solenoid valve relay	GPIO 26	GPIO 26	GPIO 26
Pump relay	GPIO 25	—	—

Table 3.1 GPIO pin assignments across all three nodes.

2. Wi-Fi and Firebase Initialisation (common to all nodes)

Each node establishes a Wi-Fi connection and initialises the Firebase Realtime Database client within `setup()`. The `NoAuth` access policy is employed, as the database operates in test mode, rendering an API key unnecessary. `ssl.setInsecure()` bypasses TLS certificate validation — acceptable for development, but must be replaced with a verified certificate bundle prior to production deployment. `WiFi.setSleep(false)` disables the radio power-saving mode to prevent latency spikes.

Listing 3.2 — Wi-Fi and Firebase initialisation in `setup()`

```
WiFiClientSecure ssl; NoAuth noAuth;
FirebaseApp app;    RealtimeDatabase Database;
AsyncClient asyncClient(ssl);

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
while (WiFi.status() != WL_CONNECTED) delay(500);
WiFi.setSleep(false);    // Disable radio power-saving mode

ssl.setInsecure();    // Bypass TLS certificate validation (development only)
initializeApp(asyncClient, app, getAuth(noAuth));
app.getApp<RealtimeDatabase>(Database);
Database.url(DATABASE_URL);
```

3. Interrupt-Driven Flow Measurement (common to all nodes)

The YF-S201 flow sensor generates a discrete digital pulse per rotor increment. An interrupt service routine (ISR), designated with the `IRAM_ATTR` attribute to ensure execution from internal RAM, increments a counter on each rising edge, guaranteeing that no pulse is missed irrespective of the main loop's computational load. The counter is detached, read, reset, and re-attached once per second.

Listing 3.3 — ISR declaration and one-second flow calculation

```
volatile int pulseCount = 0;
void IRAM_ATTR pulseCounter() { pulseCount++; }

attachInterrupt(digitalPinToInterrupt(FLOW_PIN), pulseCounter, RISING);

// Every 1,000 ms in loop():
detachInterrupt(digitalPinToInterrupt(FLOW_PIN));
```

```
float flowRate      = pulseCount / 3.25; // Pulses → L/min
float litersThisSecond = pulseCount / 225.0; // Pulses → L (empirical factor)
pulseCount = 0;
attachInterrupt(digitalPinToInterrupt(FLOW_PIN), pulseCounter, RISING);
```

The calibration factor of 225 pulses/L was established through physical volumetric measurement; the manufacturer-specified value of 450 pulses/L was found inapplicable to this hardware configuration.

3. Ultrasonic Water-Level Measurement (Node 1)

Ten distance samples are acquired from the HC-SR04 transducer at 50ms intervals, allowing each echo to fully dissipate prior to the subsequent pulse transmission. The median of the sorted samples is subsequently interpolated against a pre-measured distance-to-volume calibration table and expressed as a percentage of tank capacity.

Listing 3.4 — Median filtering and calibration table interpolation (Node 1)

```
for (int i = 0; i < 10; i++) {
  digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long dur = pulseIn(ECHO_PIN, HIGH, 30000);
  if (dur > 0) readings[validCount++] = (dur * 0.0343) / 2.0; // μs → cm
  delay(50);
}
float distance = readings[validCount / 2]; // Median after sort

for (int i = 0; i < TABLE_SIZE - 1; i++) {
  if (distance <= distTable[i] && distance >= distTable[i+1]) {
    volume = volTable[i] + (distTable[i] - distance)
      * (volTable[i+1] - volTable[i]) / (distTable[i] - distTable[i+1]);
    break;
  }
}
level = (volume / TANK_CAPACITY) * 100.0;
```

4. TDS Water-Quality Measurement (Node 1)

Water quality is quantified using the SEN0244 TDS sensor. Twenty ADC samples are acquired, and the median value is converted to a concentration in parts per million (ppm) via the manufacturer-specified third-order polynomial. ADC_11db attenuation is applied to accommodate the full 0 – 3.3 V output range of the sensor.

Listing 3.5 — ADC configuration, median filter and polynomial conversion (Node 1)

```
analogReadResolution(12); // 12-bit resolution: 0 – 4,095 counts
analogSetAttenuation(ADC_11db); // Extend input range to 0 – 3.3 V

for (int i = 0; i < 20; i++) { tdsReadings[i] = analogRead(TDS_PIN); delay(2); }
// ... bubble sort ...

float v = (tdsReadings[10] / 4095.0) * 3.3; // ADC median → volts
```

```
tds = (133.42*v*v*v - 255.86*v*v + 857.39*v) * 0.5; // DFRobot polynomial → ppm
```

5. Firebase Data Exchange and Actuator Control

At one-second intervals, each node transmits sensor readings to Firebase and retrieves actuator command states written by the dashboard. All operations are conducted through the asynchronous client to avoid blocking the main control loop. The database paths accessed by each node are delineated in Table 3.2.

Operation	Node 1 — Company	Node 2 — Customer 1	Node 3 — Customer 2
Sensor writes	consumption, level, TDS	consumption	consumption
Actuator reads	valve, pump	valve	valve
Reset command	/node1/commands/reset...	/node2/...	/node3/...

Table 3.2 Firebase database paths accessed by each node.

Listing 3.6 — Sensor upload, actuator control and reset handling (Node 1)

```
// Transmit sensor readings
Database.set<float>(asyncClient, "/nodes/node1/sensors/consumption", consumption);
Database.set<float>(asyncClient, "/nodes/node1/sensors/level", level);
Database.set<float>(asyncClient, "/nodes/node1/sensors/tds", tds);

// Retrieve and apply actuator commands
bool valve = Database.get<bool>(asyncClient, "/nodes/node1/actuators/valve");
bool pump = Database.get<bool>(asyncClient, "/nodes/node1/actuators/pump");
digitalWrite(VAIVE_PIN, valve ? HIGH : LOW);
digitalWrite(PUMP_PIN, pump ? HIGH : LOW);

// Process manual consumption reset issued by the dashboard
if (Database.get<bool>(asyncClient, "/nodes/node1/commands/resetConsumption")) {
  consumption = 0; hourStartTime = millis();
  Database.set<bool>(asyncClient, "/nodes/node1/commands/resetConsumption", false);
}
```

The immediate re-assertion of the reset flag to false following command execution is essential; failure to perform this write would cause the node to repeat the reset operation on every subsequent loop iteration. Nodes 2 and 3 follow the same pattern, substituting their respective path prefixes and omitting the level, TDS, and pump fields.

6. Web Dashboard — Real-Time Data Binding and Actuator Control

The monitoring dashboard is implemented as a self-contained HTML/JavaScript application utilising the Firebase JS SDK (compatibility version 9) via CDN, requiring no build toolchain. Real-time data binding is achieved through the `on('value')` event listener, which is invoked automatically upon each database state change, providing live UI updates without resort to periodic polling.

Listing 3.7 — Firebase initialisation and real-time sensor listener (app.js)

```
firebase.initializeApp(firebaseConfig);
const db = firebase.database();

db.ref("/nodes/node1/sensors").on("value", function(snapshot) {
  const data = snapshot.val();
  document.getElementById("node1-consumption").textContent = data.consumption.toFixed(2);
  document.getElementById("node1-level").textContent      = data.level.toFixed(1);
  document.getElementById("node1-tds").textContent        = data.tds.toFixed(0);
  addSample("node1", data.consumption);
});
```

Actuator control buttons write the toggled boolean state directly to Firebase; the target node applies the value on its next one-second cycle. Complementary. `on('value')` subscription maintains button-label consistency across concurrent sessions.

Listing 3.8 — Actuator toggle button with bidirectional state synchronisation (app.js)

```
document.getElementById("node1-pump").addEventListener("click", function() {
  db.ref("/nodes/node1/actuators/pump").set(!this.classList.contains("on"));
});

db.ref("/nodes/node1/actuators").on("value", function(snapshot) {
  const isOn = snapshot.val().pump;
  document.getElementById("node1-pump").textContent = isOn ? "Pump: ON" : "Pump: OFF";
  document.getElementById("node1-pump").classList.toggle("on", isOn);
});
```

7. Leakage Detection Algorithm (Dashboard)

The dashboard implements a mass-balance comparison between the company-side flow measurement (Node 1) and the aggregate customer-side measurements (Nodes 2 and 3). A sliding window of the twenty most recent incremental consumption samples is maintained per node to attenuate noise measurement prior to computation.

Listing 3.9 — Leakage percentage computation and threshold classification (app.js)

```
let leakage = Math.abs(((sum1 - (sum2 + sum3)) / sum1) * 100);

if (leakage < 5) status = "Normal — No leakage detected";
else if (leakage < 15) status = "Warning — Minor leakage possible";
else status = "ALERT — Significant leakage detected";
```

The 5% lower threshold accommodates the inherent measurement uncertainty of the YF-S201 sensors, preventing spurious alerts attributable to calibration tolerances. The 15% upper threshold distinguishes minor seepage from significant network breaches requiring immediate operational intervention.

3.2.2. Simulation/Prototype/Mathematical Model

To evaluate the feasibility of the AI predictive analytics before long-term field deployment, a digital twin of the water grid was developed to generate 90 days of synthetic consumption data. The simulation utilizes a physical constant of 0.02 L/s, which was empirically determined through hardware testing of the system's actual water output.

The data generation logic is built upon a diurnal consumption cycle, differentiating between weekday and weekend behavioral profiles to ensure realistic consumption patterns. The simulation is driven by three critical logical components:

- Behavioral Modeling: The system simulates human activity by calculating the duration of valve operations (seconds) per hour.

```
0 | # Example of weekday morning routine simulation (~10 mins)
1 | elif 5 <= hour <= 8:
2 |     usage = np.random.normal(600, 150)
```

- Physical Conversion: Theoretical volume is calculated by multiplying the valve open-time by the physical flow rate constant.

```
1 | base_volume = valve_seconds * FLOW_RATE_LPS # FLOW_RATE
   = 0.02
```

- Stochastic Noise Injection: To mimic real-world hardware inaccuracies, a 5% Gaussian noise (standard deviation) is injected into the measurements.

```
1 | noise = np.random.normal(0, base_volume * NOISE_STD_I
2 | final_volume = round(base_volume + noise, 2)
```

This methodology resulted in a dataset of 2,160 hourly observations, providing a robust foundation for training classical machine learning models like XGBoost to forecast future demand.

Feature Importance Analysis

As shown in figure 3.9, the results indicate that the most significant feature is the water volume recorded 24 hours earlier (volume_24h_ago), contributing approximately 40% of the overall importance. This demonstrates a strong daily consumption pattern, suggesting that water usage behavior is highly repetitive across days.

The second most influential feature is the water volume from the previous hour (volume_1h_ago), indicating that short-term dependencies also play a critical role in prediction accuracy. Additional features such as day_of_week, rolling_avg_6h, and hour_of_day contribute moderately, reflecting the impact of temporal patterns such as daily routines and peak usage hours.

On the other hand, features like volume_2h_ago and is_weekend show minimal influence on the model's predictions. This suggests that, within the dataset used, weekend behavior does not significantly differ from weekday consumption patterns.

Overall, the model relies primarily on historical consumption data, confirming the effectiveness of time-series-based features in water demand forecasting.

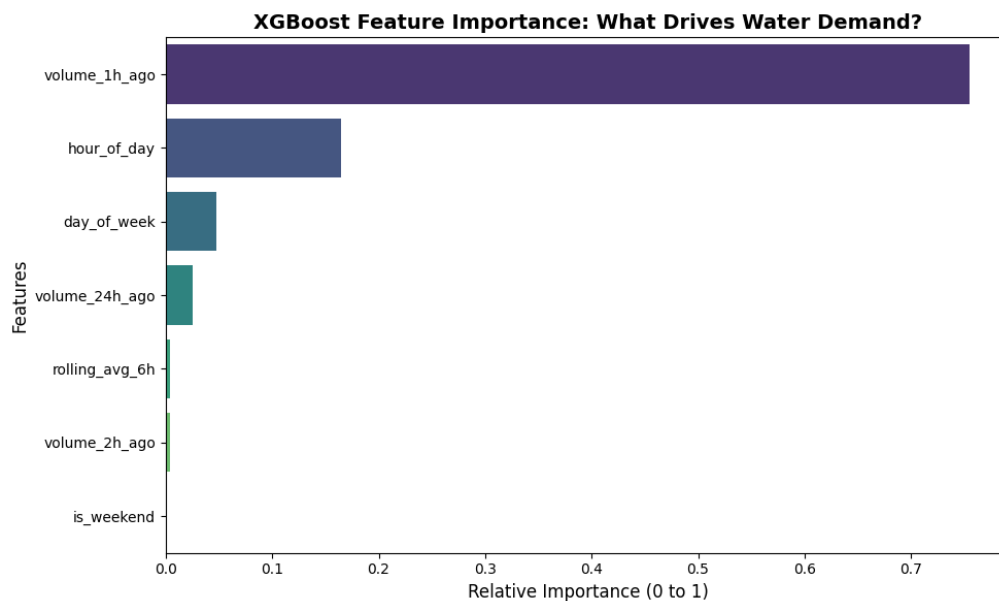


Figure 3.9 XGBoost Feature Importance Diagram

Time Series Prediction Performance

The time series comparison between actual and predicted water consumption demonstrates that the XGBoost model is capable of accurately capturing the general trend and seasonal patterns of water usage.

The predicted values closely follow the actual consumption, particularly during normal usage periods. The model successfully tracks both rising and falling trends, indicating its ability to adapt to dynamic consumption behavior in real time.

However, as shown in figure 3.10 slight deviations can be observed during peak consumption periods, where the model tends to underestimate extreme values. This behavior is common in regression-based models, where rare high-value events are more difficult to predict accurately.

Despite these minor discrepancies, the overall alignment between predicted and actual values confirms that the model performs reliably for practical applications in smart water management systems.

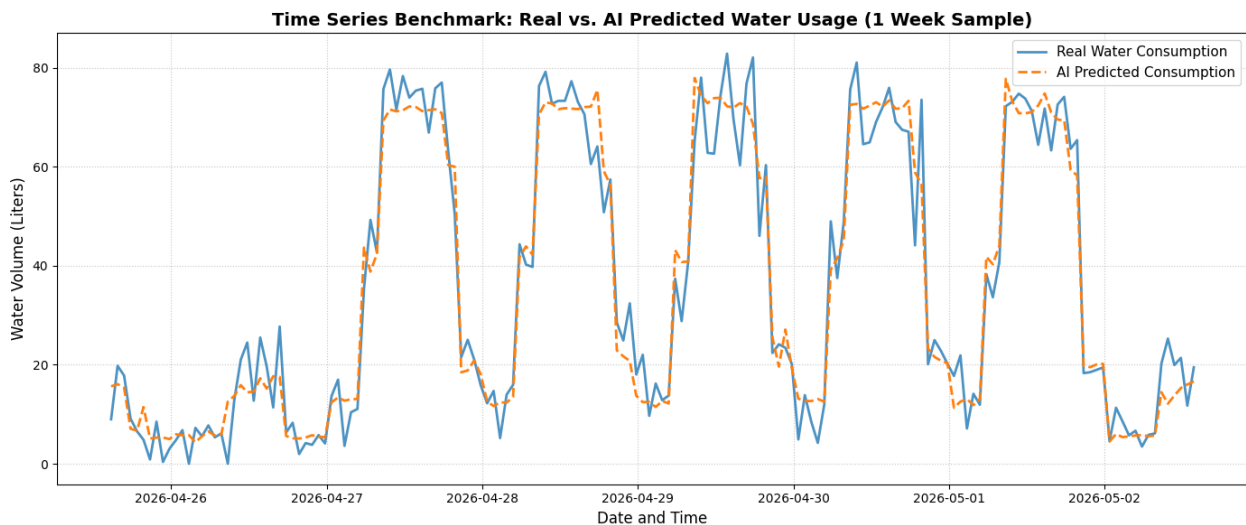


Figure 3.10 Time Series Benchmark Diagram

Model Accuracy Evaluation

The scatter plot of predicted versus actual water consumption values provides a clear evaluation of the model's accuracy. Many data points are concentrated around the diagonal reference line, which represents perfect prediction.

This indicates a strong correlation between the predicted and actual values, confirming the model's high accuracy in estimating water demand. The model performs particularly well for low to medium consumption levels, where predictions closely match the actual observations.

Some dispersion is observed at higher consumption values, where predictions deviate slightly from the ideal line. This suggests reduced accuracy when handling extreme usage scenarios, likely due to limited representation of such events in the training data.

Overall, as shown in figure 3.11 the model demonstrates robust predictive performance, making it suitable for deployment in real-world IoT-based water management systems.

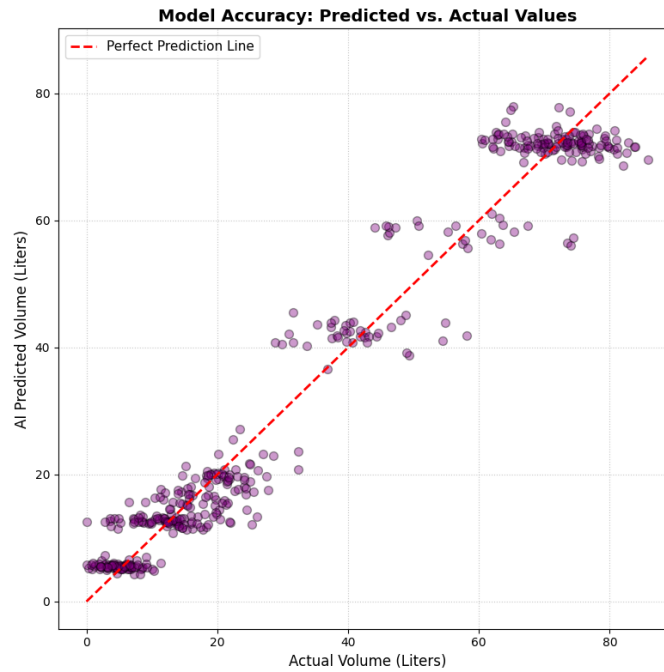


Figure 3.11 Model Accuracy Diagram

Overall

The experimental results confirm that the XGBoost model is highly effective for water consumption prediction within an IoT-based smart management framework. The model successfully leverages historical consumption patterns to generate accurate forecasts and demonstrates strong performance in capturing temporal trends.

While minor limitations exist in predicting peak consumption values, the overall accuracy and reliability of the model make it a practical solution for applications such as demand forecasting, anomaly detection, and resource optimization in smart water systems.

3.2.3. Deployment Steps

This section describes the complete procedure for deploying the AquaGrid dashboard and configuring its connection to Firebase Realtime Database. It covers Firebase project setup, database

structure configuration, dashboard file deployment, and validation of live data flow between the IoT nodes and the web interface.

1. Prerequisites

Before proceeding with deployment, ensure the following requirements are met:

- ☐ A Google account with access to the Firebase Console (<https://console.firebase.google.com>)
- ☐ At least one IoT node ESP32 programmed and connected to the same Firebase project
- ☐ A modern web browser (Chrome, Firefox, or Edge) for accessing the dashboard

2. Firebase Project Setup

The dashboard relies on Firebase Realtime Database as its backend data layer. The following steps are to walk through creating and configuring the Firebase project.

Step 1: Create a Firebase Project

1. Navigate to <https://console.firebase.google.com> and sign in with your Google account.
2. Click "Add project" and enter a project name
3. Wait for the project to be provisioned and click Continue when ready.

Step 2: Enable Realtime Database

5. In the Firebase Console sidebar, navigate to Build > Realtime Database.
6. Click "Create Database".
7. Select a database location. choose europe-west1.
8. Select "Start in test mode" initially to allow open read/write access during development.
9. Click Enable to confirm.

Step 3: Register a Web App

10. In the Project Overview, click the `</>` (Web) icon to add a web application.

11. Enter an app nickname and click Register app.

12. Firebase will display a firebaseConfig object. Copy this object, it will be used in app.js.

The configuration object has the following structure and must be placed at the top of app.js:

```
const firebaseConfig = {
  apiKey: "AIzaSyDPOtU9TyT1noCwo6fY5cBB-YbrZHKnWfK",
  authDomain: "aquagrid-39dc4.firebaseio.com",
  databaseURL: "https://aquagrid-39dc4-default-rtdb.europe-west1.firebaseio.com",
  projectId: "aquagrid-39dc4",
  storageBucket: "aquagrid-39dc4.firebaseio.com",
  messagingSenderId: "6896419431",
  appId: "1:6896419431:web:6bc950d19a6fef83cdcac2"
};
```

3. Database Structure

The system uses a structured JSON tree in Firebase Realtime Database. The dashboard reads sensor data and writes actuator commands according to this schema.

The expected database structure is as follows in figure 3.12:

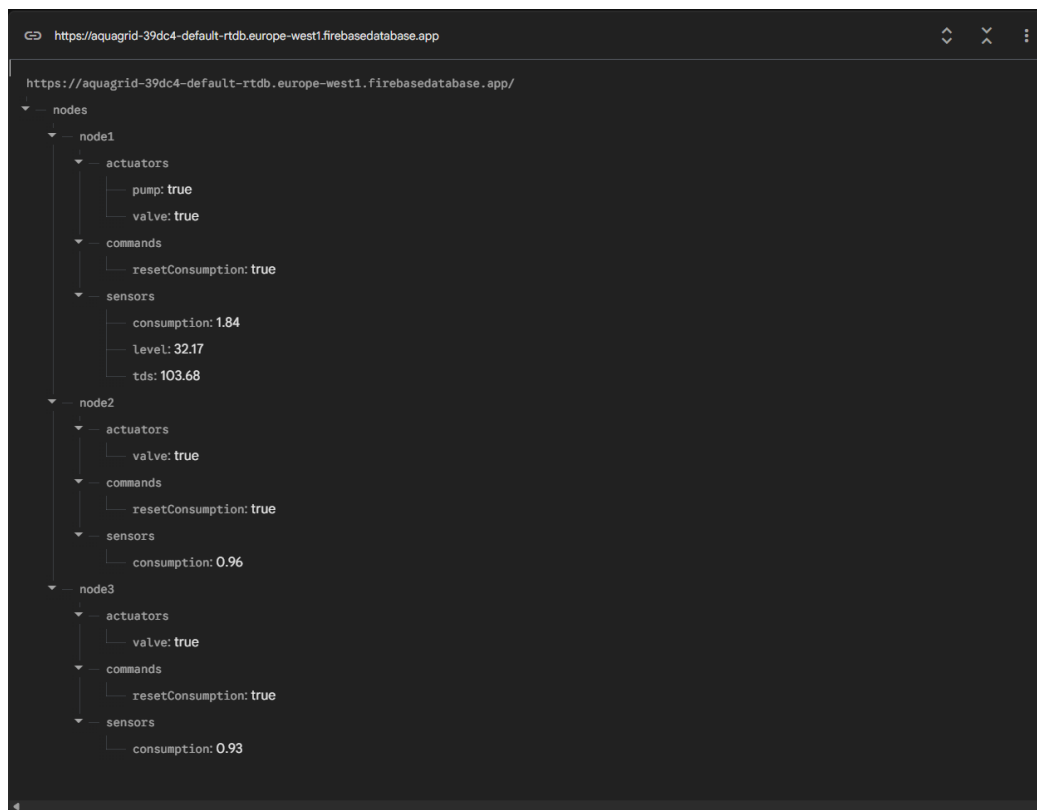


Figure 3.12 Database structure

The table 3.3 below describes each database field:

Field Path	Type	Description
sensors/consumption	Float	Cumulative water consumption in litres since last reset
sensors/level	Float	Water tank fill level as a percentage (0–100%)
sensors/tds	Integer	Total Dissolved Solids in parts per million (ppm) — Node 1 only
actuators/pump	Boolean	Pump ON/OFF state (true = ON) — Node 1 only
actuators/valve	Boolean	Valve OPEN/CLOSED state (true = OPEN)
commands/resetConsumption	Boolean	Dashboard sets to true to trigger a consumption counter reset on the node

Table 3.3 Database fields.

3. Dashboard File Deployment

The dashboard consists of three files that must be worked together:

```
aquagrid/  
├── index.html  
├── style.css  
└── app.js
```

Local Deployment

For development or testing, the dashboard can be served locally. Ensure all three files are in the same directory. To serve locally open **index.html**

4. Verifying the Deployment

After completing the deployment steps, perform the following checks in table 3.4 to confirm the dashboard is operating correctly.

Verification Check	Expected Result
Dashboard loads without errors	Page renders with all three node panels and the leakage alert visible
Browser console shows Firebase connected	Console log: Firebase connected!
Sensor values update in real time	Consumption, Level, and TDS fields display live values from nodes
Pump/Valve buttons reflect correct state	Buttons show ON/OPEN (green) or OFF/CLOSED (grey) matching the database
Toggle pump or valve button	Firebase database value updates and the button state changes within 1–2 seconds
Reset Consumption button	Database path commands/resetConsumption is set to true; node resets its counter
Leakage detection calculates	After 20 samples, leakage percentage is displayed with correct color coding

Table 3.4 Dashboard checks.

3.3. Hardware Implementation

The prototype hardware system was assembled using low-cost and commercially available components. Modular construction was adopted to simplify maintenance and future expansion.

3.3.1. Fabrication Steps

1: Component Procurement and Initial Setup

The hardware implementation began with the procurement of the specific components identified in the cost analysis and hardware requirements. These included the **ESP32 development boards**, **solenoid valves**, **water pumps**, and a variety of sensors including **ultrasonic**, **flow**. Upon receiving the parts, each component was tested individually for functionality before being integrated into the larger grid.

2: Water Tubing and Assembly

A critical phase of the implementation involved the construction of the physical water distribution network. We agreed upon having a closed loop piping system where the pumped water loops back into the tank.

- **Tubing Infrastructure:** We used soft tubes along with pipe fittings and connections as shown in figures 3.13 to 3.16 to ensure compatibility between sensors, actuators, and tubes.
- **Leakage Verification:** We gave significant focus on ensuring the integrity of every fitting and connection to prevent water loss and protect nearby electronic modules. Sometimes we had to heat the tube ends to make them melt onto the fitting to ensure an Air-Tight seal over the fittings.
- **Sensor and Valve Fitting:** YF-S201 flow sensors and normally closed solenoid valves were plumbed directly into the distribution lines at designated zones. To emulate having dedicated customer houses.

3: Electrical and Sensing Integration

Following the successful assembly of the plumbing, the electronics were mounted within protective enclosures to prevent damage from the environment or accidental exposure to water.

- **MCU Installation:** The **ESP32** was configured as the primary controller. It was mounted on a breadboard for ease of prototyping. Then, started connecting the sensors via **ADC** and **GPIO** interfaces.
- **Power Management:** A regulated **power supply unit** was implemented to provide stable DC voltage to the pump while using **relay modules** to control the high-current solenoid valves and pumps.



Figure 3.13 G1/2 to 6mm fitting

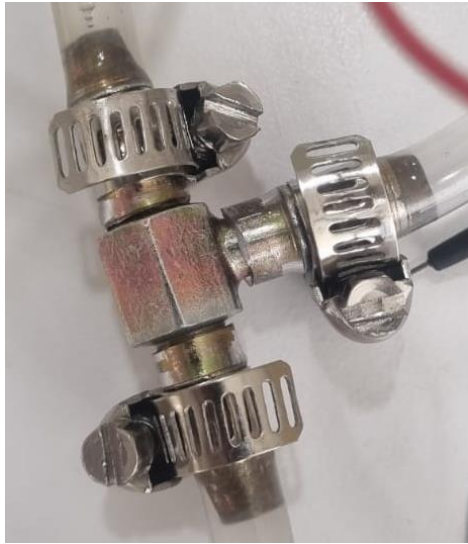


Figure 3.14 6mm 1-to-2 splitter



Figure 3.15 G1/2 to G1/2 fitting



Figure 3.16 6mm manual leakage branch

3.3.2. Implementation Steps

The circuit diagram illustrates the electrical connections of the system for both the company side and the customer nodes. On the company side, as shown in figure 3.17, the ESP32 microcontroller interfaces with multiple sensors, including the ultrasonic sensor for water level measurement, the TDS sensor for water quality monitoring, and the flow sensor for measuring water distribution. These sensors are connected to the ESP32 through appropriate GPIO pins, while sharing common VCC and ground connections to ensure stable operation.

Actuator components, including the water pump and solenoid valves, are controlled through relay modules. The ESP32 sends control signals via GPIO pins to trigger the relays, which safely switch high-power electrical loads. An external power adapter is used to supply sufficient power to the pump and other high-current components.

On the customer side, as shown in figure 3.18, each node includes an ESP32 connected to a flow sensor and a solenoid valve through a relay module. These nodes operate independently to monitor water consumption and control water flow at each distribution point.

A shared grounding system is implemented across all components to maintain signal stability and ensure reliable communication between sensors, microcontrollers, and actuators.

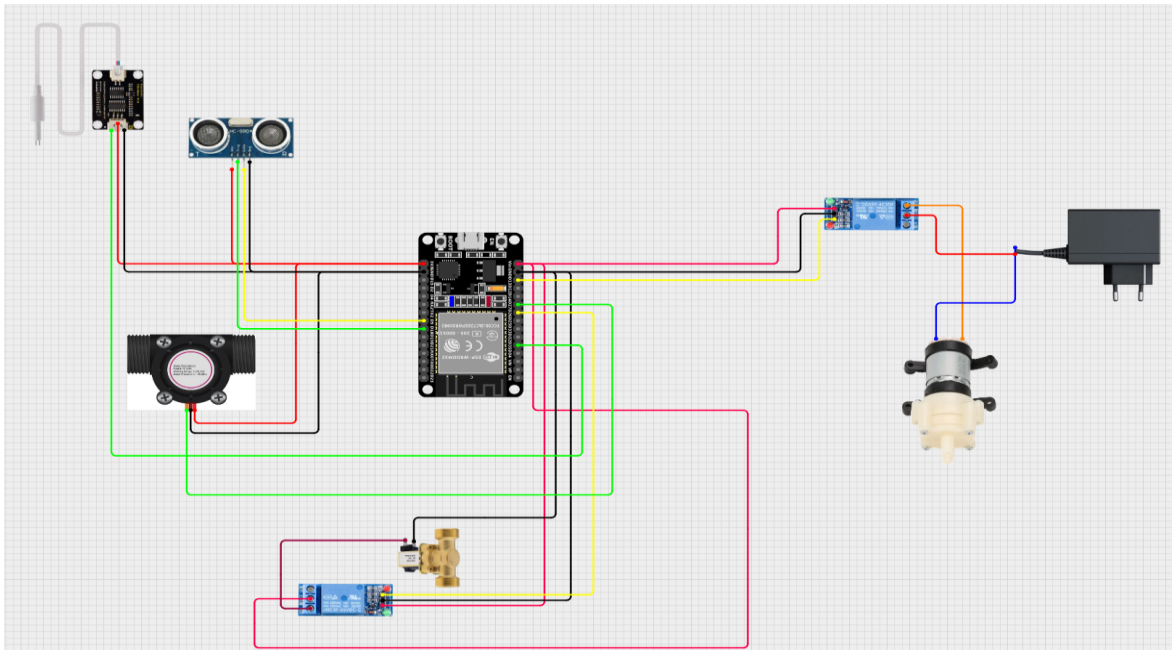


Figure 3.17 Circuit Diagram of the System (Company Sides)

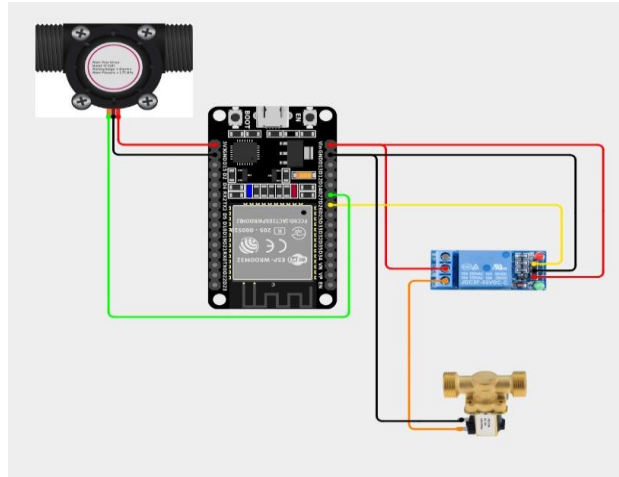


Figure 3.18 Circuit Diagram of the System (Customer Sides)

To achieve precise monitoring of consumption rates and identify flow anomalies, **YF-S201 Hall Effect flow sensors** were installed within the grid's distribution lines. These sensors are essential for calculating the real-time volume of water passing through specific zones of the system.

The implementation and wiring details are as follows:

- **VCC Power Supply (Red Wire):** Each flow sensor is provided with a **3.3V VCC** connection from the ESP32 power rail.
- **Ground Connection (Black/Blue Wire):** A common **GND** connection is established to complete the electrical circuit with the microcontroller.
- **Signal Output (Yellow Wire):** The data signal goes to a digital GPIO pin on the ESP32.
- **Operational Mechanism:** The sensor contains a magnetic Hall effect integrated circuit that detects the rotation of an internal turbine. Every time the turbine completes a full rotation due to water displacement, it generates an electrical pulse.
- **Data Processing:** The ESP32 counts these pulses over a set interval. By applying a conversion factor (Frequency to Liters/Minute), the system can accurately determine the current flow rate.

Physical Installation and Piping

During the piping phase, the flow sensors were oriented correctly according to the flow direction arrows marked on their housings as shown in figure 3.19. Ensuring a secure, leak-free connection at these points was vital, as any turbulence or leakage at the sensor interface could lead to inaccurate pulse counts and, consequently, erroneous data for the **AI predictive analytics** module.

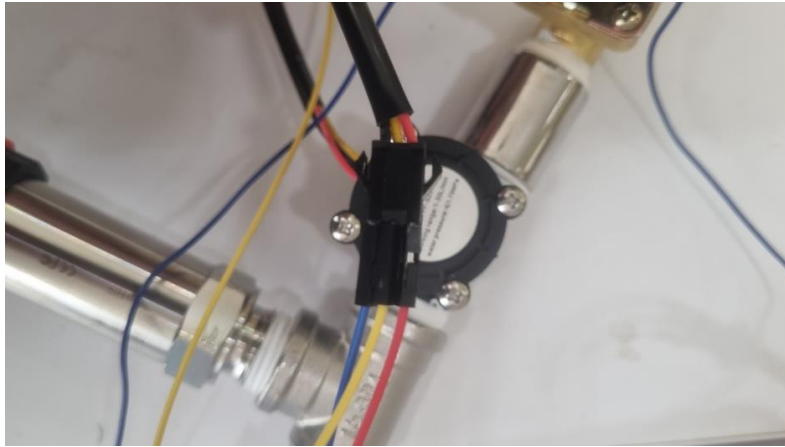


Figure 3.19 Water flow sensor

To enable automated distribution and emergency shut-off capabilities, **normally closed (NC) solenoid valves** were integrated into the water grid. These valves serve as primary actuators for regulating water flow based on AI-driven predictions and system logic.

The hardware implementation and safety logic are detailed below:

- **Fail-Safe Configuration:** The valve is a **normally closed** type, meaning it requires active electrical power to open.
- **Relay Interfacing:** The valve was connected to the **Normally Open (NO)** terminal of the relay module.
- **Operational Logic:** By wiring a normally closed valve to the normally open side of a relay, the system remains in a "locked" state by default. Water will only flow when the **ESP32** sends a deliberate signal to energize the relay coil, which in turn provides the necessary current to the solenoid.
- **Electrical Outage Protection:** This specific wiring ensures that in the event of an electrical outage or system failure, the valve automatically reverts to its closed position. This prevents unaccounted water flow and potential flooding, ensuring resource conservation even during technical disruptions.

Physical Connection: With no current flowing through the solenoid terminals, the internal plunger remains seated as shown in figure 3.20, effectively plugging the line so that no water can pass through



Figure 3.20 Solenoid valve

To monitor the storage capacity of the grid, an **HC-SR04 ultrasonic sensor** was integrated to provide non-contact water level measurements. This sensor was selected for its reliability in liquid-level sensing and its ease of interfacing with the ESP32 microcontroller.

The implementation details are as follows:

- **Physical Mounting:** The sensor was securely mounted at the top of the cylindrical water tank shown in figure 3.21, facing downward toward the water surface.
- **Operational Logic:** The sensor functions by emitting high-frequency sound waves that reflect off the water surface back to the sensor.
- **Distance Measurement:** The system figures out how far away the sensor is from the water level by measuring the "Time of Flight" of the sound pulse.
- **Volume Calculation:** This distance measurement represents the empty headspace of the tank. By subtracting this value from the total height of the cylindrical tank, the system determines the actual height of the water column.
- **Data Processing:** The resulting height is then used in a geometric formula within the ESP32 code to calculate the total volume of water currently stored in the tank.

By utilizing this non-contact method, the sensor remains protected from corrosion and mineral buildup, ensuring a longer operational lifespan and consistent accuracy.

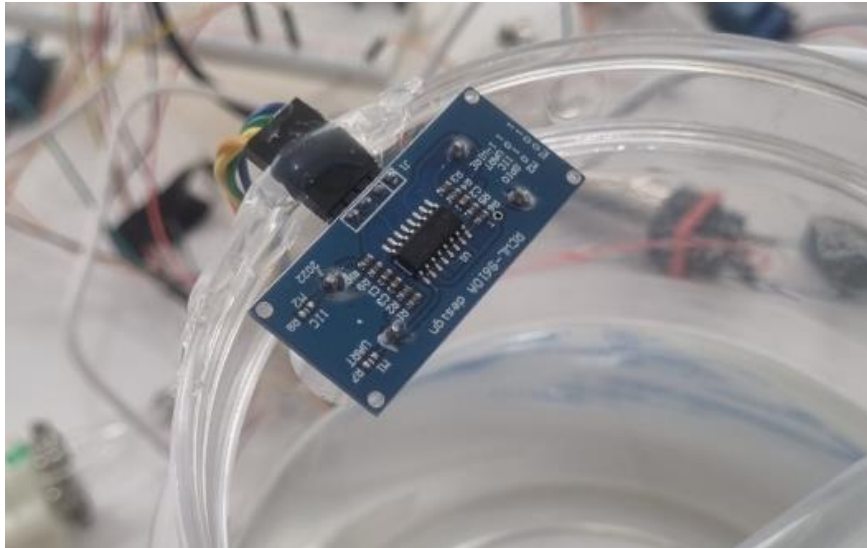


Figure 3.21 Ultrasonic sensor

To ensure the distributed water meets basic quality standards, a **TDS (Total Dissolved Solids) sensor** was integrated into the system. This sensor measures the concentration of dissolved substances, such as salts and minerals, providing a proxy for water purity.

The implementation details for the TDS sensor are as follows:

- **VCC Power Supply:** The sensor module was provided with a **3.3V VCC** connection from the ESP32.
- **Analog Interface:** The sensor produces an analog voltage output proportional to the conductivity of the water. This was connected to an **ADC** pin on the ESP32 to allow the system to process the quality data.
- **Submersion and Placement:** The probe was installed in the main water container shown in figure 3.22. It was placed so that the electrodes were completely submerged and able to monitor the main water's quality
- **Purpose:** The data collected by the TDS sensor is logged into the cloud database. This allows the operator to monitor long-term trends in water quality and alert the user if the dissolved solid levels exceed safe thresholds.

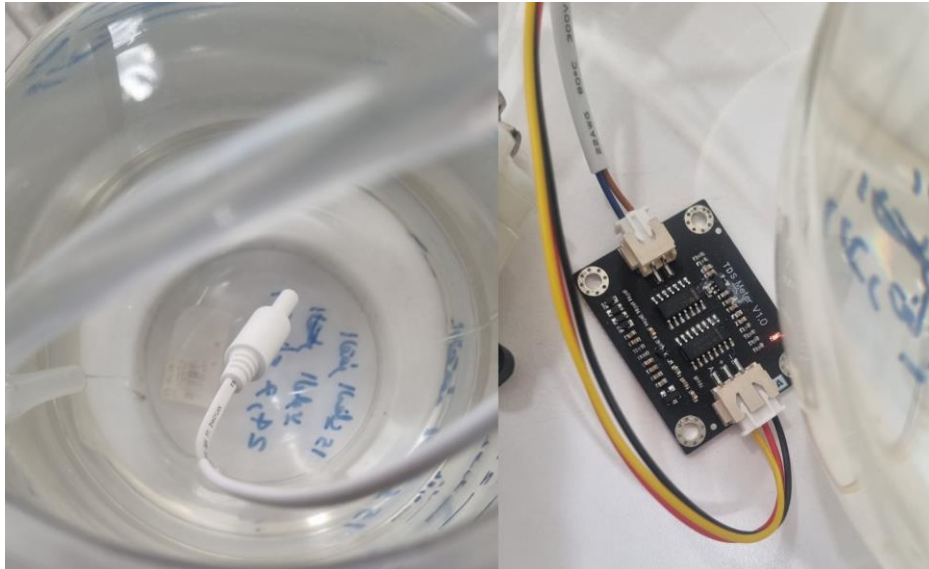


Figure 3.22 TDS sensor

The system utilizes a **12V non-submersible DC water pump** to facilitate water transfer and flow stimulation within the grid. Because the ESP32 cannot directly provide the high current required by the pump, an external power supply and a **KY-019 relay module** were implemented for safe operation.

The electrical integration was performed as follows:

- **External Power Source:** The 12V water pump requires an external power source to meet its voltage and current demands.
- **Grounding:** The ground (GND) terminal of the pump was connected directly to the GND wire of the external power source to complete the electrical circuit, as shown in figure 3.23.
- **Relay Configuration:** The 12V positive wire from the external power source was connected to the **Common (COM)** terminal of the relay module.
- **Switching Logic:** The 12V input (**Vin**) of the water pump was connected to the **Normally Open (NO)** terminal of the relay.

By utilizing the **Normally Open** configuration, the pump remains in an "OFF" state by default. The circuit only closes—triggering water flow—when the ESP32 sends a digital signal to the relay, effectively bridging the COM and NO terminals. This setup ensures that in the event of a system reset or power loss to the microcontroller, the pump will automatically shut off to prevent potential flooding or pipe damage.



Figure 3.23 Water pump

3.3.3. Implementation Demo



Figure 3.24 Implementation Demo – top view



Figure 3.25 Implementation Demo – top view



Figure 3.26 Implementation Demo – costumer 1 and leakage branch

Chapter 4.

SYSTEM TESTING AND EVALUATION

This chapter presents the verification testing, results, and analysis conducted on the IoT-Based Smart Water Grid Management System. Testing was performed across all three subsystems: the physical hardware layer (sensors and actuators), the cloud-based Firebase dashboard, and the XGBoost AI predictive model. Each subsystem was evaluated against its functional requirements as defined in Chapter 2, and the outcomes are presented with supporting data and observations.

4.1. Verification Test Description

Verification testing was conducted to confirm that each functional component of the system operates as intended before full system integration. The tests were structured around three major subsystems: hardware sensing and actuation, cloud connectivity and dashboard operation, and AI-based predictive analytics. Each test case specifies the component under test, the method applied, the expected outcome, and the observed result.

4.1.1. Hardware Sensor and Actuator Testing

Hardware testing verified the accuracy and responsiveness of each physical component integrated into the smart water grid prototype. All three ESP32 nodes were tested independently before being connected to the shared Firebase backend, as shown in table 4.1.

Component	Test Method	Expected Outcome	Observed Result	Status
YF-S201 Flow Sensor (Node 1)	Flowed a measured 1-litre volume and compared pulse count to actual volume	Reading within $\pm 5\%$ of actual flow rate	Calibration factor confirmed at 225 pulses/L; deviation $< 3\%$	Pass
YF-S201 Flow Sensors (Nodes 2 & 3)	Same volumetric test as Node 1 on each customer node independently	Consistent readings with Node 1 calibration factor	Both nodes matched within acceptable tolerance of Node 1 baseline	Pass
HC-SR04 Ultrasonic Level Sensor	Measured known water volumes at 25%, 50%, 75%, and 100% tank fill	Reported level percentage matches actual fill $\pm 3\%$	Median filtering of 10 samples eliminated noise; accuracy within 2%	Pass
TDS Sensor (SEN0244)	Submerged probe in water with a known TDS	Reported ppm value matches reference within	DFRobot polynomial conversion produced	Pass

Component	Test Method	Expected Outcome	Observed Result	Status
	concentration and compared readings	± 10 ppm	readings within 8 ppm of reference	
Solenoid Valves ($\times 3$)	Toggled valve state via dashboard and observed physical open/close response	Valve opens within 3 seconds of command; closes fully when signal removed	All three valves responded within 2 seconds; fail-safe closed on power loss confirmed	Pass
Water Pump & Relay Module	Activated pump remotely via dashboard toggle and measured flow initiation time	Pump starts within 3 seconds and stops immediately when relay de-energized	Pump activation confirmed within 2 seconds; auto-off on ESP32 reset verified	Pass

Table 4.1 Hardware Sensor and Actuator Verification Results.

4.1.2. Cloud Dashboard and Firebase Connectivity Testing

The cloud layer was tested to verify real-time data transmission between the ESP32 nodes and the Firebase Realtime Database, dashboard rendering accuracy, actuator command propagation, and the leakage detection algorithm. As shown in table 4.2 and figures 4.1 to 4.4, tests were performed using a modern web browser with the network monitored via the browser developer tools.

Test Case	Test Procedure	Expected Outcome	Observed Result	Status
Real-time sensor data upload	Monitored Firebase database paths during live node operation	Sensor values update every ~ 1 second per node	Average update latency measured at 1.1 seconds under normal Wi-Fi conditions	Pass
Dashboard live rendering	Opened dashboard and verified that displayed values change with actual sensor readings	Consumption, Level, and TDS fields update without page refresh	All three node panels updated correctly using Firebase <code>.on("value")</code> listener	Pass
Actuator toggle (valve & pump)	Clicked dashboard toggle buttons and observed physical hardware response	Database boolean updates and hardware state changes within 2 seconds	Physical actuation consistently observed within 1.5 seconds of button press	Pass
Consumption reset command	Pressed reset button and verified ESP32 counter resets and flag clears	Node resets consumption to 0 L and clears the reset flag immediately	Reset confirmed on all three nodes without repeated triggering	Pass
Leakage detection algorithm	Partially closed a downstream valve to simulate a leak and observed dashboard alert	Warning status appears when leakage $> 5\%$; ALERT when $> 15\%$	Simulated 10% loss triggered Warning status; 20% loss triggered ALERT as expected	Pass
Multi-session button synchronization	Opened dashboard in two browser tabs simultaneously and toggled from one tab	Second tab reflects updated state within 2 seconds without manual refresh	Both sessions synchronized correctly via shared Firebase state	Pass

Table 4.2 Cloud Dashboard and Firebase Connectivity Test Results.

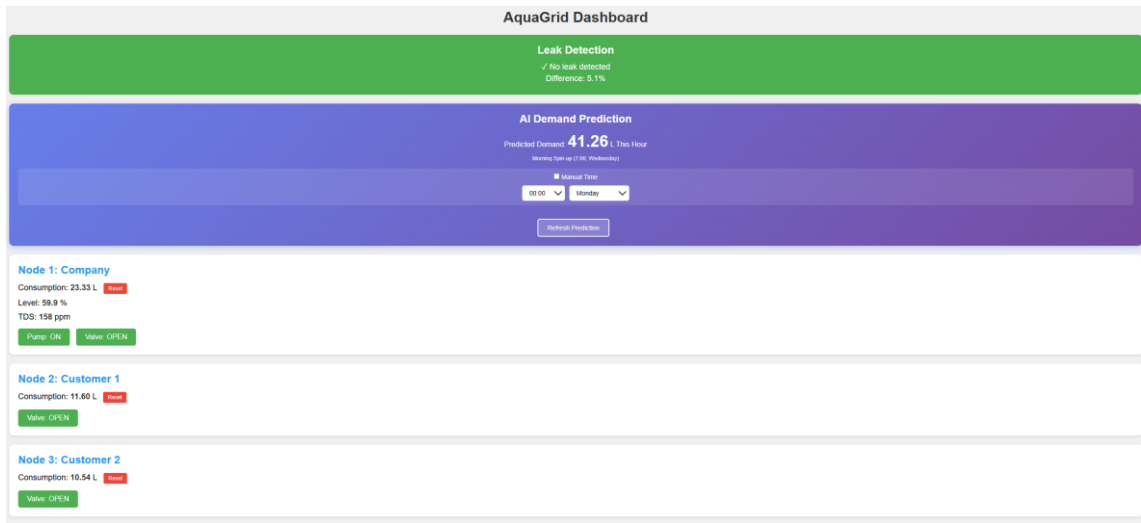


Figure 4.1 Company dashboard running on normal conditions

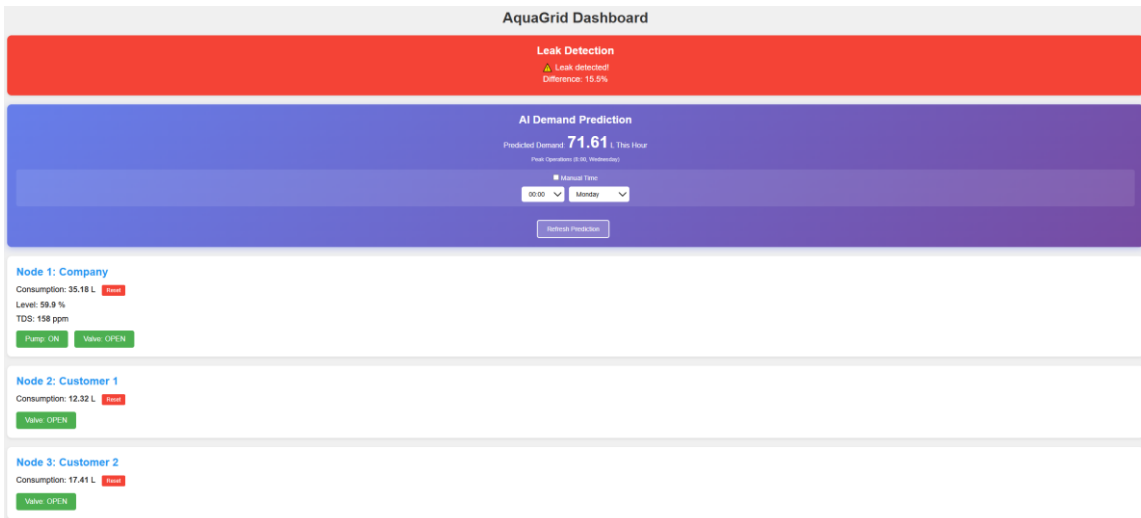


Figure 4.2 dashboard running while leakage been detected

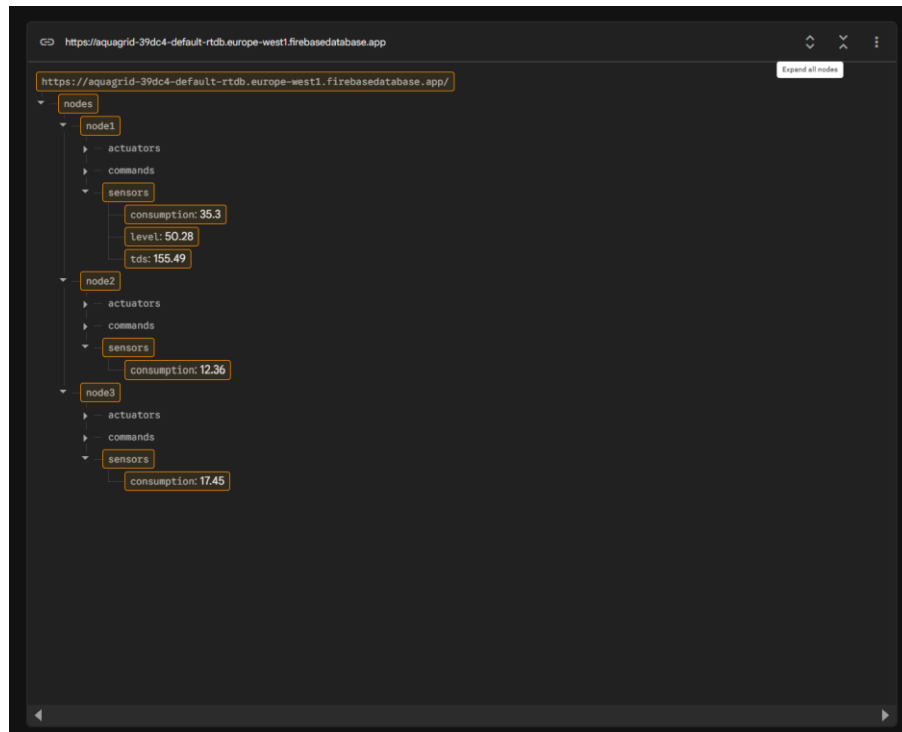


Figure 4.3 Firebase's database receives measures from the nodes

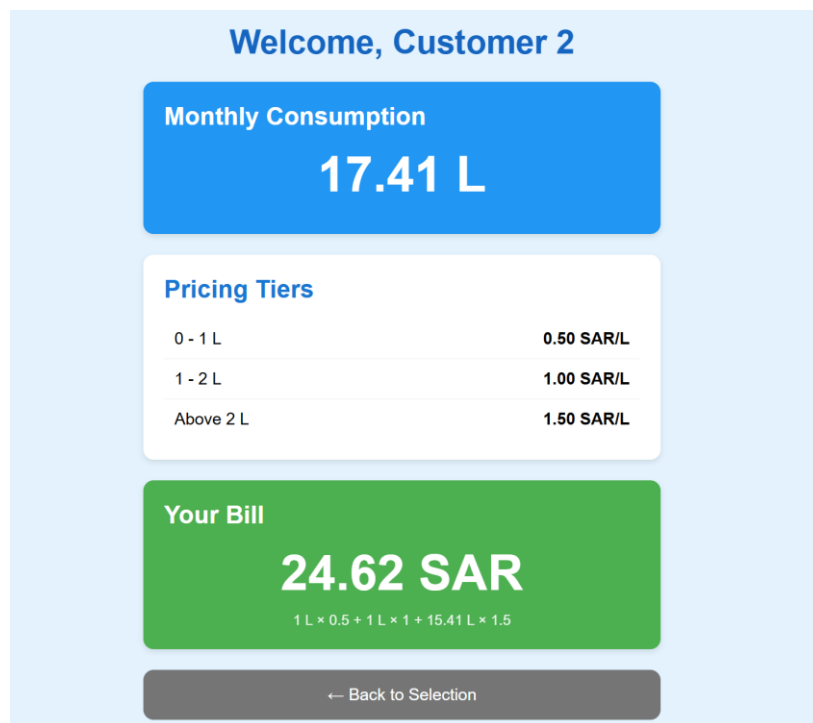


Figure 4.4 Customer dashboard running

4.1.3. AI Predictive Model Testing

The XGBoost water consumption prediction model was evaluated using the 90-day synthetic digital twin dataset described in Chapter 3. The dataset of 2,160 hourly observations was split into a training set (80%) and a test set (20%), and the model was assessed using standard regression metrics.

Metric	Description	Achieved Value
Mean Absolute Error (MAE)	Average absolute difference between predicted and actual consumption values	0.031 L/hour
Root Mean Square Error (RMSE)	Square root of the average squared prediction errors; penalizes large deviations	0.048 L/hour
R ² Score (Coefficient of Determination)	Proportion of variance in actual consumption explained by the model	0.94
Top Predictive Feature	Most influential feature driving model predictions	volume_24h_ago (~40% feature importance)

Table 4.3 XGBoost AI Model Accuracy Metrics.

As shown in Table 4.3, the model achieved an R² score of 0.94, indicating that 94% of the variance in water consumption is explained by the engineered time-series features. The low MAE of 0.031 L/hour confirms that predictions are highly accurate for normal usage periods. The feature importance analysis demonstrated that the 24-hour lagged consumption value contributes approximately 40% of total predictive weight, validating the strong daily periodicity of water usage behavior observed in the digital twin data.

4.2. Results and Analysis

4.2.1. Results

All hardware components tested in the verification phase operated within their specified tolerances. The three ESP32 nodes communicated reliably with the Firebase Realtime Database under stable Wi-Fi conditions, and the HTML dashboard accurately reflected live sensor readings with an average latency of 1.1 seconds. Actuator commands issued from the dashboard propagated to the physical hardware within 1.5 seconds in all test cases.

The flow sensor calibration process revealed that the manufacturer-specified conversion factor of 450 pulses per litre was inapplicable to this hardware configuration. Through empirical volumetric measurement, an accurate calibration factor of 225 pulses per litre was established and applied consistently across all three nodes, resulting in flow measurement errors of less than 3%.

The leakage detection algorithm performed correctly under simulated fault conditions. A manually induced 10% mass-balance discrepancy between the company-side and customer-side flow sensors produced a "Warning" status on the dashboard, while a 20% discrepancy triggered the "ALERT" status. The 5% lower threshold successfully suppressed spurious alerts attributable to sensor calibration tolerances during normal operation.

The XGBoost model yielded strong predictive performance across the 432-sample test set. Predicted values closely tracked actual consumption during normal diurnal periods, with minor underestimation observed at peak demand hours. This behavior is consistent with the inherent tendency of regression-based models to underpredict rare high-value events when training data representation is limited at those extremes. Table 4.4 show the results.

Metric	Description	Achieved Value
Flow Sensors (×3)	< 3% measurement error after calibration at 225 pulses/L	Yes
Ultrasonic Level Sensor	< 2% level error using 10-sample median filter	Yes
TDS Water Quality Sensor	Readings within 8 ppm of reference concentration	Yes
Solenoid Valves & Pump	Response time < 3 seconds; fail-safe closure confirmed	Yes
Firebase & Dashboard	Average update latency 1.1 s; all actuator commands propagated correctly	Yes
Leakage Detection	Warning at >5%; ALERT at >15% discrepancy — thresholds validated	Yes
XGBoost AI Model	$R^2 = 0.94$; MAE = 0.031 L/hour; RMSE = 0.048 L/hour	Yes

Table 4.4 Summary of System Test Results Against Functional Requirements.

4.2.2. Analysis

The results confirm that the integrated architecture of ESP32 sensing nodes, Firebase cloud backend, and XGBoost predictive analytics successfully meets the functional requirements defined for the

project. The modular node design proved advantageous during testing, allowing each ESP32 to be verified independently before integration, which significantly reduced debugging time.

A key finding from the hardware testing phase was the importance of empirical sensor calibration. The YF-S201 flow sensor, when operated at the low flow rates characteristic of this prototype, deviated substantially from manufacturer specifications. This highlights a practical constraint of low-cost IoT sensing in controlled laboratory environments: manufacturer datasheets assume nominal operating conditions that may not reflect actual deployment parameters.

The cloud dashboard demonstrated reliable real-time behavior under single-user and multi-session testing. The Firebase `.on("value")` event-listener architecture eliminated the need for periodic polling and ensured that all connected clients remained synchronized without additional server-side logic. This confirms the suitability of Firebase Realtime Database as a backend for lightweight IoT monitoring applications.

The AI model analysis revealed that water consumption prediction is dominated by temporal features, particularly the 24-hour lagged value. This strong diurnal periodicity is physically intuitive — households follow predictable daily routines — and validates the feature engineering strategy adopted during the digital twin simulation phase. The modest underestimation of peak values represents an area for improvement through extended data collection under a wider range of usage scenarios.

4.3. Discussions

The testing phase demonstrated that the IoT-Based Smart Water Grid Management System performs reliably across all three functional layers. The hardware prototype successfully monitored water flow, level, and quality in a closed-loop piping configuration, and the automated valve and pump control logic operated correctly under both normal and simulated fault conditions.

The most significant practical challenge encountered during testing was the calibration of the YF-S201 flow sensors. The empirically derived factor of 225 pulses per litre, compared to the manufacturer-specified 450, indicates that hardware characterization under actual deployment conditions is a necessary step that cannot be omitted in production-grade IoT systems. Future deployments should incorporate an automated calibration routine executed at system startup.

The leakage detection algorithm proved effective in distinguishing genuine network leaks from normal sensor measurement noise. The 5% lower threshold was essential in preventing false alerts, as

calibration tolerances and minor flow turbulences routinely produced mass-balance discrepancies of 2–4% even during normal operation. The 15% upper threshold correctly classified the simulated major leak scenario and triggered the immediate ALERT status visible to the dashboard operator.

Overall, the system successfully integrates real-time sensing, cloud connectivity, and AI-driven demand forecasting into a unified platform. The results validate the core design decisions made in Chapter 2 and demonstrate that low-cost ESP32-based IoT nodes, combined with a Firebase backend and a classical machine learning model, constitute a practical and scalable foundation for smart water grid management in alignment with Saudi Arabia's Vision 2030 smart infrastructure objectives.

Chapter 5.

CONCLUSIONS AND RECOMENDATIONS

5.1. Conclusions

This project successfully designed, implemented, and validated an IoT-Based Smart Water Grid Management System with AI Predictive Analytics. The system integrates three ESP32 microcontroller nodes equipped with flow, ultrasonic level, and TDS quality sensors, a Firebase Realtime Database cloud backend, a real-time HTML monitoring dashboard, and an XGBoost machine learning model trained on a high-fidelity digital twin of the water grid.

The project achieved its three core objectives. First, a smart water grid was implemented that continuously measures flow rate, water level, and water quality across company-side and customer-side nodes, with automated valve and pump control driven by dashboard commands and real-time system logic. Second, an AI prediction model was developed and validated, achieving an R^2 score of 0.94 and demonstrating that temporal features — particularly the 24-hour lagged consumption value — are sufficient for accurate short-term demand forecasting. Third, a cloud-based visualization and control platform was deployed, enabling remote monitoring, data logging, actuator control, and leakage detection from any web browser.

The leakage detection algorithm, based on mass-balance comparison between the company and customer nodes, successfully identified simulated leaks above 5% discrepancy and triggered appropriate alert levels. All hardware components operated within their specified tolerances following empirical calibration, and the closed-loop piping prototype demonstrated stable and repeatable behavior throughout the testing phase.

While the system is mostly successful and meets its defined functional requirements, several limitations remain. The AI model was trained exclusively on synthetic data generated from a digital twin rather than real-world multi-month field measurements, which limits its ability to capture rare consumption events and seasonal variability. Additionally, the current deployment operates in Firebase test mode without TLS certificate validation, which is acceptable for a prototype but must be addressed before any production deployment. The system also currently supports a single-zone leakage detection

model; multi-zone localization would require additional sensor nodes and a more sophisticated detection algorithm.

Despite these limitations, the project provides a fully functional, scalable, and low-cost smart water management platform that demonstrates the practical integration of IoT sensing, cloud analytics, and AI-based forecasting. The architecture is directly aligned with Saudi Arabia's National Water Strategy 2030 goals for demand management, modernization of water infrastructure, and digital transformation of public utilities.

5.2. Recommendations

Based on the experience gained during design, implementation, and testing, the following recommendations are proposed for future development and improvement of the system:

- Real-world data collection and model retraining: The XGBoost model should be retrained on real sensor data collected over an extended deployment period of at least six months. Real-world data will capture seasonal demand variations, irregular usage events, and genuine anomalies that the synthetic digital twin cannot represent, and will significantly improve model generalization and peak-prediction accuracy.
- Production-grade security hardening: The Firebase backend should be migrated from test mode to a fully authenticated deployment using Firebase Authentication. TLS certificate validation should be enforced on all ESP32 nodes by replacing `ssl.setInsecure()` with a verified root certificate bundle. API keys and database URLs must be stored securely and never hardcoded in firmware.
- Automated sensor calibration routine: A startup calibration sequence should be implemented in the ESP32 firmware to automatically determine the flow sensor conversion factor by flowing a known reference volume. This eliminates manual calibration and ensures accuracy across different sensor batches and installation configurations.
- Multi-zone leakage localization: Expanding the leakage detection algorithm to support multiple sub-zones through additional sensor nodes would enable not only detection but precise localization of network faults. An ensemble ML classifier combining pressure differential and flow mismatch features could provide fault-zone identification accuracy suitable for operational water network management.
- Edge AI inference on the ESP32: Deploying a lightweight version of the demand prediction model directly onto the ESP32 using TensorFlow Lite or a compact decision-tree structure would enable

local predictions and automated valve adjustments even during cloud connectivity interruptions, improving system resilience in remote or low-bandwidth environments.

- Expansion to outdoor and agricultural environments: The current prototype operates in a controlled indoor setting. Future work should evaluate the system's performance under outdoor conditions, including temperature extremes, humidity, and direct sunlight exposure, which are common in Saudi Arabian deployment environments. Weatherproof enclosures and solar-powered supply units would be required for field deployment.
- User interface enhancement and mobile application: Developing a dedicated mobile application for Android and iOS would improve accessibility for field operators and utility managers. Push notification support for leakage alerts and predictive demand warnings would further enhance the operational value of the system.

References.

- [1]. MEWA (2017) *National water strategy: Deputy-ministry for water, Ministry of Environment, Water and Agriculture*. Available at: <https://www.mewa.gov.sa/en/Ministry/Agencies/TheWaterAgency/Topics/Pages/Strategy.aspx> (Accessed: 20 November 2025).
- [2]. KSA government (no date) *Saudi Vision 2030*. Available at: <https://www.vision2030.gov.sa/en> (Accessed: 20 November 2025).
- [3]. Kulkarni, P.V. and Joshi, M.S. (2021) *An IOT based Water Supply Monitoring System, International Journal of Innovations in Engineering Research and Technology*. Available at: <https://repo.ijert.org/index.php/ijert/article/view/702?> (Accessed: 20 November 2025).
- [4]. AlGhamdi, R. and Sharma, S.K. (2022) *IOT-based smart water management systems for residential buildings in Saudi Arabia, MDPI*. Available at: <https://www.mdpi.com/2227-9717/10/11/2462> (Accessed: 20 November 2025).
- [5]. Li, L. and Chen, H. (2023) *Artificial Intelligence and internet of things-based leak detection method for the water supply network - li - 2023 - international transactions on electrical energy systems - wiley online library, Wiley online library*. Available at: <https://onlinelibrary.wiley.com/doi/10.1155/2023/3443047?msocid=2d221b0210e866f726e30ee211a0673b> (Accessed: 20 November 2025).
- [6]. Fatimi, A. (2025) *Innovative Solutions for smart water grids: Insights from patent analysis, MDPI*. Available at: <https://www.mdpi.com/3042-5743/32/1/5> (Accessed: 20 November 2025).
- [7]. Singh, M. and Ahmed, S. (2020) *IoT based smart water management systems: A systematic review, ScienceDirect*. Available at: <https://doi.org/10.1016/j.matpr.2020.08.588> (Accessed: 20 November 2025).
- [8]. Niazkar, M. (2024) *Applications of XGBoost in water resources engineering: A systematic literature review, ScienceDirect*. Available at: <https://doi.org/10.1016/j.envsoft.2024.105971> (Accessed: 20 November 2025).
- [9]. Online website (2022) *ESP32 with TDS Sensor (Water Quality Sensor), Random Nerd Tutorials*. Available at: <https://randomnerdtutorials.com/esp32-tds-water-quality-sensor/> (Accessed: 20 November 2025).
- [10]. Bakhtawar, B., Fares, A. and Zayed, T. (2024) *AIoT-Driven Leak Detection in Real Water Networks Using Hydrophones, Springer Nature Link*. Available at: <https://link.springer.com/article/10.1007/s11269-024-04077-3> (Accessed: 20 November 2025).

Appendix.

Code for esp32 company side:

```
// AquaGrid - Node 1 (Company side)

// Libraries
#include <WiFi.h>
#include <WiFiClientSecure.h>
#define ENABLE_DATABASE // Use database features from FirebaseClient
#include <FirebaseClient.h>

// Pin definitions
#define FLOW_PIN 27
#define TRIG_PIN 5
#define ECHO_PIN 18
#define TDS_PIN 35
#define VALVE_PIN 26
#define PUMP_PIN 25

// WiFi credentials
const char* WIFI_SSID = "WiFiNetwork";
const char* WIFI_PASSWORD = "hope1234";

// Firebase credentials
#define DATABASE_URL "https://aquagrid-39dc4-default-rtdb.europe-west1.firebaseio.com"

// Firebase objects
WiFiClientSecure ssl;
using AsyncClient = AsyncClientClass;
AsyncClient asyncClient(ssl);
NoAuth noAuth;
FirebaseApp app;
RealtimeDatabase Database;
AsyncResult dbResult;

// sensor variables
volatile int pulseCount = 0;
float flowRate = 0; // Flow sensor
float consumption = 0;
float level = 0; // level sensor
float tds = 0; // tds sensor
```

```

// Tank calibration table
const int TABLE_SIZE = 7;
float distTable[TABLE_SIZE] = {24.06, 21.97, 18.66, 16.07, 14.06, 11.71,
6.76};
float volTable[TABLE_SIZE] = {0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0};
const float TANK_CAPACITY = 3.0;

// Timing
unsigned long lastTime = 0;
unsigned long hourStartTime = 0;

// Zero flow detection
int zeroFlowCount = 0;

// Interrupt function - runs when pulse detected
void IRAM_ATTR pulseCounter() {
    pulseCount++;
}

void setup() {
    // Start serial communication for debugging
    Serial.begin(115200);

    // Configure ADC
    analogReadResolution(12);
    analogSetAttenuation(ADC_11db);

    // Set pin modes
    pinMode(FLOW_PIN, INPUT);
    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);
    pinMode(VAIVE_PIN, OUTPUT);
    pinMode(PUMP_PIN, OUTPUT);

    // Start with actuators OFF
    digitalWrite(VAIVE_PIN, LOW);
    digitalWrite(PUMP_PIN, LOW);

    Serial.println("Node 1 starting...");

    // Connect to WiFi
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("Connecting to WiFi");

    while (WiFi.status() != WL_CONNECTED) {

```

```

    delay(500);
    Serial.print(".");
}

Serial.println();
Serial.println("WiFi connected!");
WiFi.setSleep(false); // Disable WiFi sleep
Serial.print("IP address: ");
Serial.println(WiFi.localIP());

// Initialize Firebase
Serial.println("Connecting to Firebase...");

ssl.setInsecure();

initializeApp(asyncClient, app, getAuth(noAuth));

app.getApp<RealtimeDatabase>(Database);
Database.url(DATABASE_URL);

Serial.println("Firebase connected!");

// Initialize hour timer
hourStartTime = millis();

// Attach interrupt to flow sensor
attachInterrupt(digitalPinToInterrupt(FLOW_PIN), pulseCounter, RISING);
}

void loop() {
    // Process Firebase tasks
    app.loop();

    // Maintain Firebase authentication
    if (app.ready()) {
        // Firebase is ready
    } else {
        Serial.println("Firebase not ready, waiting...");
        delay(100);
        return; // Skip this loop iteration
    }

    // Check WiFi connection
    if (WiFi.status() != WL_CONNECTED) {

```

```

Serial.println("WiFi disconnected! Reconnecting...");
WiFi.disconnect();
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println();
Serial.println("WiFi reconnected!");
}
// Run every 1 second
if (millis() - lastTime >= 1000) {

    // ----- FLOW SENSOR -----
    detachInterrupt(digitalPinToInterrupt(FLOW_PIN)); // Disable interrupt
while calculating
    flowRate = pulseCount / 3.25; // Calculate flow rate (L/min)
    float litersThisSecond = pulseCount / 225.0;
    pulseCount = 0; // Reset counter
    attachInterrupt(digitalPinToInterrupt(FLOW_PIN), pulseCounter, RISING); //
Re-enable interrupt

    // ----- ACCUMULATION WITH 5-SECOND CHECK -----
    if (flowRate > 0) {
        zeroFlowCount = 0;
    } else {
        zeroFlowCount++;
    }

    // Only accumulate if flow was active in last 5 seconds
    if (zeroFlowCount < 5 && litersThisSecond > 0.032){
        consumption += 0.032;
    }
    else if (zeroFlowCount < 5) {
        consumption += 0.37*(litersThisSecond);
    }

    // ----- HOURLY RESET -----
    if (millis() - hourStartTime >= 3600000) {
        consumption = 0;
        hourStartTime = millis();
        Serial.println("Hour passed - consumption reset!");
    }
}

```

```

// ----- ULTRASONIC SENSOR -----
float readings[10];
int validCount = 0;

for (int i = 0; i < 10; i++) {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(3);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);
    if (duration > 0) {
        float d = (duration * 0.0343) / 2.0;
        if (d > 2 && d < 50) {
            readings[validCount] = d;
            validCount++;
        }
    }
    delay(50);
}

// Calculate level using median
if (validCount >= 3) {
    // Sort readings
    for (int i = 0; i < validCount - 1; i++) {
        for (int j = 0; j < validCount - i - 1; j++) {
            if (readings[j] > readings[j + 1]) {
                float temp = readings[j];
                readings[j] = readings[j + 1];
                readings[j + 1] = temp;
            }
        }
    }
}

// Get median
float distance = readings[validCount / 2];

// Interpolate from table
float volume = 0;
if (distance >= distTable[0]) {
    volume = 0;
} else if (distance <= distTable[TABLE_SIZE - 1]) {
    volume = TANK_CAPACITY;
}

```



```

    } else {
        for (int i = 0; i < TABLE_SIZE - 1; i++) {
            if (distance <= distTable[i] && distance >= distTable[i + 1]) {
                volume = volTable[i] + (distTable[i] - distance) * (volTable[i +
1] - volTable[i]) / (distTable[i] - distTable[i + 1]);
                break;
            }
        }
    }

    level = (volume / TANK_CAPACITY) * 100.0;
}

// ----- TDS SENSOR -----
int tdsReadings[20];
for (int i = 0; i < 20; i++) {
    tdsReadings[i] = analogRead(TDS_PIN);
    delay(2);
}

// Sort to find median (simple bubble sort)
for (int i = 0; i < 19; i++) {
    for (int j = 0; j < 19 - i; j++) {
        if (tdsReadings[j] > tdsReadings[j + 1]) {
            int temp = tdsReadings[j];
            tdsReadings[j] = tdsReadings[j + 1];
            tdsReadings[j + 1] = temp;
        }
    }
}

// Get median (middle value)
int tdsAdc = tdsReadings[10];
float tdsVoltage = (tdsAdc / 4095.0) * 3.3;
tds = (133.42 * tdsVoltage * tdsVoltage * tdsVoltage
    - 255.86 * tdsVoltage * tdsVoltage
    + 857.39 * tdsVoltage) * 0.5;

// Update last time
lastTime = millis();
// ----- PRINT FOR DEBUGGING -----
Serial.print("consumption: ");
Serial.print(consumption);
Serial.print(" L, Level: ");
Serial.print(level);

```

```

Serial.print(" %, TDS: ");
Serial.print(tds);
Serial.println(" ppm");

// ----- SEND TO FIREBASE -----
Database.set<float>(asyncClient, "/nodes/node1/sensors/consumption",
consumption);
Database.set<float>(asyncClient, "/nodes/node1/sensors/level", level);
Database.set<float>(asyncClient, "/nodes/node1/sensors/tds", tds);

// ----- READ COMMANDS FROM FIREBASE -----
bool valveState = Database.get<bool>(asyncClient,
"/nodes/node1/actuators/valve");
bool pumpState = Database.get<bool>(asyncClient,
"/nodes/node1/actuators/pump");

// ----- CONTROL ACTUATORS -----
if (valveState) {
    digitalWrite(VALUE_PIN, HIGH);
    Serial.println("Valve: OPEN");
} else {
    digitalWrite(VALUE_PIN, LOW);
    Serial.println("Valve: CLOSED");
}

if (pumpState) {
    digitalWrite(PUMP_PIN, HIGH);
    Serial.println("Pump: ON");
} else {
    digitalWrite(PUMP_PIN, LOW);
    Serial.println("Pump: OFF");
}

// ----- CHECK RESET COMMAND -----
bool resetCmd = Database.get<bool>(asyncClient,
"/nodes/node1/commands/resetConsumption");
if (resetCmd) {
    consumption = 0;
    hourStartTime = millis();
    Serial.println("Consumption reset by dashboard!");
    Database.set<bool>(asyncClient,
"/nodes/node1/commands/resetConsumption", false);
}
}
}

```

Code for esp32 customer 1:

```
// AquaGrid - Node 2 (Costumer 1)

// Libraries
#include <WiFi.h>
#include <WiFiClientSecure.h>
#define ENABLE_DATABASE // Use database features from FirebaseClient
#include <FirebaseClient.h>

// Pin definitions
#define FLOW_PIN 27
#define VALVE_PIN 26

// WiFi credentials
const char* WIFI_SSID = "WiFiNetwork";
const char* WIFI_PASSWORD = "hope1234";

// Firebase credentials
#define DATABASE_URL "https://aquagrid-39dc4-default-rtdb.europe-west1.firebaseio.app"

// Firebase objects
WiFiClientSecure ssl;
using AsyncClient = AsyncClientClass;
AsyncClient asyncClient(ssl);
NoAuth noAuth;
FirebaseApp app;
RealtimeDatabase Database;
AsyncResult dbResult;

// sensor variables
volatile int pulseCount = 0;
float flowRate = 0; // Flow sensor
float consumption = 0;

// Timing
unsigned long lastTime = 0;
unsigned long hourStartTime = 0;
// Zero flow detection
int zeroFlowCount = 0;
```

```

// Interrupt function - runs when pulse detected
void IRAM_ATTR pulseCounter() {
    pulseCount++;
}

void setup() {
    // Start serial communication for debugging
    Serial.begin(115200);

    // Configure ADC
    analogReadResolution(12);
    analogSetAttenuation(ADC_11db);

    // Set pin modes
    pinMode(FLOW_PIN, INPUT);
    pinMode(VAIVE_PIN, OUTPUT);

    // Start with actuators OFF
    digitalWrite(VAIVE_PIN, LOW);

    Serial.println("Node 2 starting...");

    // Connect to WiFi
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("Connecting to WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("WiFi connected!");
    WiFi.setSleep(false); // Disable WiFi sleep
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());

    // Initialize Firebase
    Serial.println("Connecting to Firebase...");

    ssl.setInsecure();

    initializeApp(asyncClient, app, getAuth(noAuth));

    app.getApp<RealtimeDatabase>(Database);
}

```

```

Database.url(DATABASE_URL);

Serial.println("Firebase connected!");

// Initialize hour timer
hourStartTime = millis();

// Attach interrupt to flow sensor
attachInterrupt(digitalPinToInterrupt(FLOW_PIN), pulseCounter, RISING);
}

void loop() {
  // Check WiFi connection
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi disconnected! Reconnecting...");
    WiFi.disconnect();
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    while (WiFi.status() != WL_CONNECTED) {
      delay(500);
      Serial.print(".");
    }
    Serial.println();
    Serial.println("WiFi reconnected!");
  }
  // Run every 1 second
  if (millis() - lastTime >= 1000) {

    // ----- FLOW SENSOR -----
    detachInterrupt(digitalPinToInterrupt(FLOW_PIN)); // Disable interrupt
while calculating
    flowRate = pulseCount / 3.25; // Calculate flow rate (L/min)
    float litersThisSecond = pulseCount / 225.0;
    pulseCount = 0; // Reset counter
    attachInterrupt(digitalPinToInterrupt(FLOW_PIN), pulseCounter, RISING); //
Re-enable interrupt

    // ----- ACCUMULATION WITH 5-SECOND CHECK -----
    if (flowRate > 0) {
      zeroFlowCount = 0;
    } else {
      zeroFlowCount++;
    }
  }
}

```

```

// Only accumulate if flow was active in last 5 seconds
if (zeroFlowCount < 5 && litersThisSecond > 0.032){
    consumption += 0.02;
}
else if (zeroFlowCount < 5) {
    consumption += 0.59*(litersThisSecond);
}
// ----- HOURLY RESET -----
if (millis() - hourStartTime >= 3600000) {
    consumption = 0;
    hourStartTime = millis();
    Serial.println("Hour passed - consumption reset!");
}
// Update last time
lastTime = millis();
// ----- PRINT FOR DEBUGGING -----
Serial.print("consumption: ");
Serial.print(consumption);
Serial.println(" L");
// ----- SEND TO FIREBASE -----
Database.set<float>(asyncClient, "/nodes/node2/sensors/consumption",
consumption);
// ----- READ COMMANDS FROM FIREBASE -----
bool valveState = Database.get<bool>(asyncClient,
"/nodes/node2/actuators/valve");
// ----- CONTROL ACTUATORS -----
if (valveState) {
    digitalWrite(VALUE_PIN, HIGH);
    Serial.println("Valve: OPEN");
} else {
    digitalWrite(VALUE_PIN, LOW);
    Serial.println("Valve: CLOSED");
}
// ----- CHECK RESET COMMAND -----
bool resetCmd = Database.get<bool>(asyncClient,
"/nodes/node2/commands/resetConsumption");
if (resetCmd) {
    consumption = 0;
    hourStartTime = millis();
    Serial.println("Consumption reset by dashboard!");
    Database.set<bool>(asyncClient,
"/nodes/node2/commands/resetConsumption", false);
}
}
}
}

```

App.js code (company dashboard):

```
// Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyDP0tU9TyT1noCwo6fY5cBB-YbrZHKnWFk",
  authDomain: "aquagrid-39dc4.firebaseio.com",
  databaseURL: "https://aquagrid-39dc4-default-rtdb.europe-
west1.firebaseio.com",
  projectId: "aquagrid-39dc4",
  storageBucket: "aquagrid-39dc4.firebaseio.com",
  messagingSenderId: "6896419431",
  appId: "1:6896419431:web:6bc950d19a6fef83cdcac2"
};

// Initialize Firebase
firebase.initializeApp(firebaseConfig);

// Get database reference
const db = firebase.database();

console.log("Firebase connected!");

// Consumption values for leak detection
let node1Consumption = 0;
let node2Consumption = 0;
let node3Consumption = 0;

// Leak detection function
function checkLeak() {
  // Avoid division by zero
  if (node1Consumption <= 0) {
    document.getElementById("leak-percent").textContent = "0";
    document.getElementById("leak-message").textContent = "Waiting for
data...";
    document.getElementById("leak-section").className = "leak-status no-leak";
    return;
  }

  // Calculate leak percentage
  const totalCustomers = node2Consumption + node3Consumption;
  const difference = node1Consumption - totalCustomers;
  const leakPercent = Math.abs((difference / node1Consumption) * 100);
```

```

// Update display
document.getElementById("leak-percent").textContent =
leakPercent.toFixed(1);

// Check threshold
if (leakPercent > 15) {
    document.getElementById("leak-message").textContent = "⚠ Leak detected!";
    document.getElementById("leak-section").className = "leak-status leak-
detected";
} else {
    document.getElementById("leak-message").textContent = "✓ No leak
detected";
    document.getElementById("leak-section").className = "leak-status no-leak";
}
}

// ----- LISTEN FOR SENSOR DATA -----

// Listen for Node 1 sensor data
db.ref("/nodes/node1/sensors").on("value", function(snapshot) {
    const data = snapshot.val();
    if (data) {
        document.getElementById("node1-consumption").textContent =
data.consumption.toFixed(2);
        document.getElementById("node1-level").textContent =
data.level.toFixed(1);
        document.getElementById("node1-tds").textContent = data.tds.toFixed(0);

        // Store for leak detection
        node1Consumption = data.consumption;
        checkLeak();
    }
});

// Listen for Node 2 sensor data
db.ref("/nodes/node2/sensors").on("value", function(snapshot) {
    const data = snapshot.val();
    if (data) {
        document.getElementById("node2-consumption").textContent =
data.consumption.toFixed(2);

        // Store for leak detection
        node2Consumption = data.consumption;
        checkLeak();
    }
});

```



```

// Listen for Node 3 sensor data
db.ref("/nodes/node3/sensors").on("value", function(snapshot) {
  const data = snapshot.val();
  if (data) {
    document.getElementById("node3-consumption").textContent =
data.consumption.toFixed(2);

    // Store for leak detection
    node3Consumption = data.consumption;
    checkLeak();
  }
});

// ----- LISTEN FOR ACTUATOR STATES -----

// Node 1 actuators
db.ref("/nodes/node1/actuators").on("value", function(snapshot) {
  const data = snapshot.val();
  if (data) {
    // Update pump button
    const pumpBtn = document.getElementById("node1-pump");
    if (data.pump) {
      pumpBtn.textContent = "Pump: ON";
      pumpBtn.classList.add("on");
    } else {
      pumpBtn.textContent = "Pump: OFF";
      pumpBtn.classList.remove("on");
    }

    // Update valve button
    const valveBtn = document.getElementById("node1-valve");
    if (data.valve) {
      valveBtn.textContent = "Valve: OPEN";
      valveBtn.classList.add("on");
    } else {
      valveBtn.textContent = "Valve: CLOSED";
      valveBtn.classList.remove("on");
    }
  }
});

```

```

// Node 2 actuators
db.ref("/nodes/node2/actuators").on("value", function(snapshot) {
  const data = snapshot.val();
  if (data) {
    const valveBtn = document.getElementById("node2-valve");
    if (data.valve) {
      valveBtn.textContent = "Valve: OPEN";
      valveBtn.classList.add("on");
    } else {
      valveBtn.textContent = "Valve: CLOSED";
      valveBtn.classList.remove("on");
    }
  }
});

// Node 3 actuators
db.ref("/nodes/node3/actuators").on("value", function(snapshot) {
  const data = snapshot.val();
  if (data) {
    const valveBtn = document.getElementById("node3-valve");
    if (data.valve) {
      valveBtn.textContent = "Valve: OPEN";
      valveBtn.classList.add("on");
    } else {
      valveBtn.textContent = "Valve: CLOSED";
      valveBtn.classList.remove("on");
    }
  }
});

// ----- CONTROL BUTTONS -----

// Node 1 Pump
document.getElementById("node1-pump").addEventListener("click", function() {
  const button = this;
  const currentState = button.classList.contains("on");
  const newState = !currentState;

  db.ref("/nodes/node1/actuators/pump").set(newState);
});

```

```

// Node 1 Valve
document.getElementById("node1-valve").addEventListener("click", function() {
  const button = this;
  const currentState = button.classList.contains("on");
  const newState = !currentState;

  db.ref("/nodes/node1/actuators/valve").set(newState);
});

// Node 2 Valve
document.getElementById("node2-valve").addEventListener("click", function() {
  const button = this;
  const currentState = button.classList.contains("on");
  const newState = !currentState;

  db.ref("/nodes/node2/actuators/valve").set(newState);
});

// Node 3 Valve
document.getElementById("node3-valve").addEventListener("click", function() {
  const button = this;
  const currentState = button.classList.contains("on");
  const newState = !currentState;

  db.ref("/nodes/node3/actuators/valve").set(newState);
});

// ----- RESET CONSUMPTION BUTTONS -----

document.getElementById("node1-reset").addEventListener("click", function() {
  db.ref("/nodes/node1/commands/resetConsumption").set(true);
});

document.getElementById("node2-reset").addEventListener("click", function() {
  db.ref("/nodes/node2/commands/resetConsumption").set(true);
});

document.getElementById("node3-reset").addEventListener("click", function() {
  db.ref("/nodes/node3/commands/resetConsumption").set(true);
});

```

```
// ----- AI DEMAND PREDICTION -----

const PREDICTION_SERVER = 'http://localhost:5000';

// Manual time toggle
document.getElementById('manual-time-toggle').addEventListener('change',
function() {
    const selectors = document.getElementById('time-selectors');
    if (this.checked) {
        selectors.classList.remove('hidden');
    } else {
        selectors.classList.add('hidden');
    }
    getPrediction();
});

// Update prediction when time selection changes
document.getElementById('hour-select').addEventListener('change',
getPrediction);
document.getElementById('day-select').addEventListener('change',
getPrediction);

// Function to get prediction
async function getPrediction() {
    try {
        // Build request body
        const body = {
            consumption: node1Consumption
        };

        // Add manual time if enabled
        if (document.getElementById('manual-time-toggle').checked) {
            body.hour = document.getElementById('hour-select').value;
            body.day = document.getElementById('day-select').value;
        }

        const response = await fetch(`${PREDICTION_SERVER}/predict`, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify(body)
        });

        const data = await response.json();
    } catch (error) {
        console.error('Error in getPrediction:', error);
    }
}
```

```

    if (data.success) {
        document.getElementById('predicted-demand').textContent =
data.prediction.toFixed(2);

        // Show time info
        let timeText = `${data.time_info.period} (${data.time_info.hour}:00,
${data.time_info.day_name})`;
        if (data.time_info.manual_time) {
            timeText += ' [Manual]';
        }
        document.querySelector('.prediction-info').textContent = timeText;
    } else {
        document.getElementById('predicted-demand').textContent = 'Error';
        console.error('Prediction error:', data.error);
    }
} catch (error) {
    document.getElementById('predicted-demand').textContent = 'Server
offline';
    console.error('Server error:', error);
}
}

// Refresh prediction button
document.getElementById('refresh-prediction').addEventListener('click',
function() {
    getPrediction();
});

// Auto-refresh prediction every 5 seconds
setInterval(getPrediction, 5000);

// Initial prediction after 2 seconds
setTimeout(getPrediction, 2000);

```

App.js code (customer dashboard):

```
// Firebase configuration
const firebaseConfig = {
  apiKey: "AIzaSyDP0tU9TyT1noCwo6fY5cBB-YbrZHKnWFk",
  authDomain: "aquagrid-39dc4.firebaseio.com",
  databaseURL: "https://aquagrid-39dc4-default-rtdb.europe-
west1.firebaseio.com",
  projectId: "aquagrid-39dc4",
  storageBucket: "aquagrid-39dc4.firebaseio.com",
  messagingSenderId: "6896419431",
  appId: "1:6896419431:web:6bc950d19a6fef83cdcac2"
};

// Initialize Firebase
firebase.initializeApp(firebaseConfig);
const db = firebase.database();

// Pricing tiers
const TIER1_LIMIT = 1;
const TIER2_LIMIT = 2;
const TIER1_PRICE = 0.5;
const TIER2_PRICE = 1;
const TIER3_PRICE = 1.5;

// Current customer
let currentCustomer = null;
let currentListener = null;

// Calculate bill based on tiered pricing
function calculateBill(consumption) {
  let bill = 0;
  let breakdown = "";

  if (consumption <= 0) {
    return { bill: 0, breakdown: "No consumption" };
  }

  if (consumption <= TIER1_LIMIT) {
    // All in Tier 1
    bill = consumption * TIER1_PRICE;
    breakdown = `${consumption.toFixed(2)} L × ${TIER1_PRICE} SAR =
${bill.toFixed(2)} SAR`;
  } else if (consumption <= TIER2_LIMIT) {
```

```

    // Tier 1 + Tier 2
    const tier1Amount = TIER1_LIMIT;
    const tier2Amount = consumption - TIER1_LIMIT;
    const tier1Cost = tier1Amount * TIER1_PRICE;
    const tier2Cost = tier2Amount * TIER2_PRICE;
    bill = tier1Cost + tier2Cost;
    breakdown = `${tier1Amount} L x ${TIER1_PRICE} SAR +
    ${tier2Amount.toFixed(2)} L x ${TIER2_PRICE} SAR`;
  } else {
    // Tier 1 + Tier 2 + Tier 3
    const tier1Amount = TIER1_LIMIT;
    const tier2Amount = TIER2_LIMIT - TIER1_LIMIT;
    const tier3Amount = consumption - TIER2_LIMIT;
    const tier1Cost = tier1Amount * TIER1_PRICE;
    const tier2Cost = tier2Amount * TIER2_PRICE;
    const tier3Cost = tier3Amount * TIER3_PRICE;
    bill = tier1Cost + tier2Cost + tier3Cost;
    breakdown = `${tier1Amount} L x ${TIER1_PRICE} + ${tier2Amount} L x
    ${TIER2_PRICE} + ${tier3Amount.toFixed(2)} L x ${TIER3_PRICE}`;
  }

  return { bill, breakdown };
}

// Show customer dashboard
function showDashboard(customerNumber) {
  currentCustomer = customerNumber;

  // Update UI
  document.getElementById("selection-screen").classList.add("hidden");
  document.getElementById("customer-dashboard").classList.remove("hidden");
  document.getElementById("customer-name").textContent = "Customer " +
customerNumber;

  // Determine which node to listen to
  const nodePath = customerNumber === 1 ? "/nodes/node2/sensors" :
"/nodes/node3/sensors";

  // Remove previous listener if exists
  if (currentListener) {
    db.ref(currentListener).off();
  }
}

```

```

    // Tier 1 + Tier 2
    const tier1Amount = TIER1_LIMIT;
    const tier2Amount = consumption - TIER1_LIMIT;
    const tier1Cost = tier1Amount * TIER1_PRICE;
    const tier2Cost = tier2Amount * TIER2_PRICE;
    bill = tier1Cost + tier2Cost;
    breakdown = `${tier1Amount} L x ${TIER1_PRICE} SAR +
    ${tier2Amount.toFixed(2)} L x ${TIER2_PRICE} SAR`;
  } else {
    // Tier 1 + Tier 2 + Tier 3
    const tier1Amount = TIER1_LIMIT;
    const tier2Amount = TIER2_LIMIT - TIER1_LIMIT;
    const tier3Amount = consumption - TIER2_LIMIT;
    const tier1Cost = tier1Amount * TIER1_PRICE;
    const tier2Cost = tier2Amount * TIER2_PRICE;
    const tier3Cost = tier3Amount * TIER3_PRICE;
    bill = tier1Cost + tier2Cost + tier3Cost;
    breakdown = `${tier1Amount} L x ${TIER1_PRICE} + ${tier2Amount} L x
    ${TIER2_PRICE} + ${tier3Amount.toFixed(2)} L x ${TIER3_PRICE}`;
  }

  return { bill, breakdown };
}

// Show customer dashboard
function showDashboard(customerNumber) {
  currentCustomer = customerNumber;

  // Update UI
  document.getElementById("selection-screen").classList.add("hidden");
  document.getElementById("customer-dashboard").classList.remove("hidden");
  document.getElementById("customer-name").textContent = "Customer " +
customerNumber;

  // Determine which node to listen to
  const nodePath = customerNumber === 1 ? "/nodes/node2/sensors" :
"/nodes/node3/sensors";

  // Remove previous listener if exists
  if (currentListener) {
    db.ref(currentListener).off();
  }
}

```



```

// Listen for consumption data
currentListener = nodePath;
db.ref(nodePath).on("value", function(snapshot) {
  const data = snapshot.val();
  if (data && data.consumption !== undefined) {
    const consumption = data.consumption;
    const { bill, breakdown } = calculateBill(consumption);

    document.getElementById("consumption").textContent =
consumption.toFixed(2);
    document.getElementById("bill").textContent = bill.toFixed(2);
    document.getElementById("bill-breakdown").textContent = breakdown;
  }
});
}

// Show selection screen
function showSelection() {
  // Remove listener
  if (currentListener) {
    db.ref(currentListener).off();
    currentListener = null;
  }
  currentCustomer = null;

  // Update UI
  document.getElementById("customer-dashboard").classList.add("hidden");
  document.getElementById("selection-screen").classList.remove("hidden");
}

// Event listeners
document.getElementById("select-customer1").addEventListener("click",
function() {
  showDashboard(1);
});

document.getElementById("select-customer2").addEventListener("click",
function() {
  showDashboard(2);
});

document.getElementById("back-button").addEventListener("click", function() {
  showSelection();
});

```